SPIRIT-FL: A Sparse Polynomial Framework for Privacy-Preserving and Verifiable Federated Learning with Independent Sparsification

***Abstract*—Federated learning (FL) enables collaborative model training without sharing local datasets, but exchanging high-dimensional model updates introduces substantial communication overhead. Top-k sparsification alleviates this problem by allowing clients to transmit only a small subset of important updates. However, independently selected sparsified model updates break the common-coordinate assumption underlying existing aggregation and verification mechanisms, introducing new challenges in privacy-preserving sparse aggregation, verifiable aggregation, and adaptive weighting. To address these challenges, we propose SPIRIT-FL, a privacy-preserving and verifiable FL framework with independent sparsification. SPIRIT-FL represents sparsified model updates as sparse polynomials, where coordinates are encoded as exponents and update values are represented as coefficients. This representation enables aggregation over independently selected sparse coordinates without revealing sparse coordinates or update values. Building on this representation, SPIRIT-FL provides a polynomial-based verification mechanism for sparse aggregation and employs blockchain-assisted Bottom-ℓ sketches to derive adaptive aggregation weights without requiring a trusted aggregation server. Security analysis demonstrates the privacy, correctness, and verifiability guarantees of SPIRIT-FL. Experimental results show that SPIRIT-FL substantially reduces computation overhead across key protocol phases, with reductions of several orders of magnitude in aggregation and verification against multiple baselines, while reducing per-client communication by up to 40-fold under the 128-bit field setting and maintaining competitive model utility.**

I. Introduction

Federated learning (FL) enables multiple distributed clients to collaboratively train a global model without directly sharing their local datasets [1], [2]. In each round, clients perform local training and upload model updates to an aggregation server. However, as modern models continue to grow in size, repeatedly transmitting dense model updates has become a major communication bottleneck in practical FL deployments [3]. Therefore, reducing communication overhead while preserving model utility remains a fundamental challenge.

To address this challenge, Top-k sparsification has been widely adopted as an effective communication reduction technique in FL [4], [5]. Instead of uploading a complete model update, each client independently retains only the kd largest-magnitude entries of its local model update, where d denotes the model dimension and k is a small Top-k sparsification ratio. Such independent sparsification can reduce communication by several orders of magnitude while maintaining competitive model utility. However, independent Top-k sparsification fundamentally changes the security and aggregation assumptions of existing FL systems [6].

Conventional secure aggregation [7], [8] and verifiable aggregation schemes [9], [10] are designed for dense model updates defined over a common coordinate space, where the corresponding entries from different clients can be directly aggregated. After independent Top-k sparsification, different clients generally retain different subsets of coordinates according to their local data distributions [11]. Consequently, aggregation is no longer a simple value summation operation. The aggregation server must first align sparse entries corresponding to the same model coordinates across clients before computing the aggregated model update [12].

This necessity for coordinate alignment also introduces a new privacy challenge. A straightforward approach is to transmit sparse coordinates to the aggregation server for alignment [13]. However, recent studies have shown that sparse coordinates themselves may leak sensitive information about clients' local training data [5], [6]. Therefore, privacy protection must be extended from model update values to sparse coordinates, creating a fundamental dilemma: coordinate alignment requires identifying corresponding sparse coordinates across clients, while the coordinates themselves should not be revealed. Existing secure aggregation protocols [12], [14] assume that client updates are defined over a shared coordinate space and therefore cannot directly support privacy-preserving aggregation of independently sparsified model updates. Existing approaches either avoid coordinate alignment by enforcing a common sparsification mask [15], limiting the flexibility and utility of independent sparsification, or rely on additional trust assumptions, such as multiple non-colluding servers [5] or trusted execution environments [16], to privately aggregate independently sparsified updates. This leads to the first challenge: *How can independently sparsified model updates be correctly aligned and aggregated without revealing sparse coordinates or model update values?*

Beyond privacy-preserving aggregation, the correctness of sparse aggregation also becomes more challenging in the presence of an untrusted aggregation server. A malicious or lazy server may skip required computations or return

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
ISBN 978-1-970672-09-1
https://dx.doi.org/10.14722/ndss.2027.[23|24]xxxx
www.ndss-symposium.org

a manipulated aggregation result to reduce its workload or influence the global model [17], [18]. In conventional FL [19], [20], verification focuses on ensuring that the returned model update is consistent with the aggregation of client model updates defined over a common coordinate space. However, independent sparsification introduces an additional verification requirement: the server must correctly align sparse coordinates before aggregating their values. Therefore, verification should cover both coordinate alignment and value aggregation rather than the final aggregation result alone. Existing verifiable FL schemes [17], [18], [19], [20] mainly focus on verifying aggregation under the common coordinate assumption, where model updates are defined over predefined coordinate spaces. They cannot directly verify independently sparsified aggregation because the correctness of aggregation additionally depends on whether sparse coordinates are properly aligned before value aggregation. This leads to the second challenge: *How can clients efficiently verify the correctness of sparse aggregation when both coordinate alignment and value aggregation may be manipulated by an untrusted server?*

Furthermore, practical FL systems often suffer from statistical heterogeneity caused by Non-IID client data. Adaptive aggregation methods improve model convergence by assigning different weights to clients according to model update similarity [21], [22], [23]. Existing approaches [24] typically rely on the aggregation server to compute client similarity and aggregation weights. This design assumes that the server faithfully performs the weighting procedure. However, an untrusted server may manipulate similarity measurements or aggregation weights to influence the resulting global model. Therefore, the weighting process should be securely derived without requiring the aggregation server to be trusted. Independently selected sparse coordinates provide structural information about client updates. Clients with similar data distributions tend to retain more similar coordinate patterns. This observation suggests that sparse coordinate structures can be used to derive aggregation weights without requiring the aggregation server to determine the weights. However, such a weighting mechanism must protect sparse coordinate privacy and prevent an untrusted server from influencing the derived weights. This leads to the third challenge: *How can adaptive aggregation weights be securely derived from sparse coordinate structures without relying on an untrusted aggregation server?*

These challenges are interdependent and cannot be effectively addressed through independent solutions. Thus, a unified representation that captures both sparse coordinates and update values is required. Our key insight is that independently sparsified model updates can be represented as sparse polynomials, where coordinates are encoded as exponents and update values are represented as coefficients. This representation provides a unified algebraic foundation for secure aggregation and verification in independently sparsified federated learning. Based on this insight, we propose **SPIRIT-FL**, a sparse-polynomial framework for privacy-preserving and verifiable aggregation of independently sparsified model updates in FL. SPIRIT-FL integrates sparse polynomial interpolation with blockchain-assisted Bottom-ℓ sketches to address the challenges of private aggregation, verifiable aggregation, and adaptive weighting under independent sparsification. The main contributions of this work are summarized as follows:

- We propose a sparse-polynomial aggregation framework for independently sparsified FL. By representing sparse coordinates as polynomial exponents and update values as coefficients, SPIRIT-FL enables coordinate alignment through polynomial aggregation while preserving the privacy of sparse coordinates and update values.
- We design a polynomial-based verification mechanism for independently sparsified aggregation. Unlike existing verifiable FL schemes that assume predefined coordinate spaces, SPIRIT-FL enables clients to verify sparse aggregation correctness, including coordinate alignment and value aggregation, even when the aggregation server colludes with up to $N-2$ clients.
- We develop an adaptive weighting mechanism for heterogeneous federated learning without relying on the aggregation server. By leveraging blockchain-assisted Bottom-ℓ sketches, SPIRIT-FL enables clients to independently derive consistent similarity-based aggregation weights while preserving sparse coordinate privacy.
- We provide rigorous security analysis and extensive performance evaluation of SPIRIT-FL. The results demonstrate that SPIRIT-FL substantially reduces communication and computation overhead compared with representative privacy-preserving and verifiable FL schemes, while maintaining competitive model utility.

II. Background

A. Federated Learning

Federated learning [1] enables multiple distributed clients to collaboratively train a global model without sharing their private local datasets. Consider an FL system consisting of N clients, denoted by $\mathcal{U} = \{\mathcal{U}_1, \mathcal{U}_2, \ldots, \mathcal{U}_N\}$, where each client $\mathcal{U}_i$ owns a private local dataset $\mathcal{D}_i$ with $|\mathcal{D}_i|$ samples. The total number of samples is denoted as $|\mathcal{D}| = \sum_{i=1}^{N} |\mathcal{D}_i|$. The global optimization objective is formulated as $\min_{\theta \in \mathbb{R}^d} F(\theta) = \sum_{i=1}^{N} p_i F_i(\theta)$, where $F_i(\theta)$ denotes the local empirical loss function of $\mathcal{U}_i$, $\theta \in \mathbb{R}^d$ represents the global model parameter, and $p_i = \frac{|\mathcal{D}_i|}{|\mathcal{D}|}$ denotes the aggregation weight of client $\mathcal{U}_i$.

The training process proceeds over T communication rounds. At round t ($0 \le t < T$), the server broadcasts the current global model θ^t to each client $\mathcal{U}_i \in \mathcal{U}$.

Local Update. Each client $\mathcal{U}_i$ initializes its local model as $\theta_i^{t,0} = \theta^t$, and performs SGD over E local epochs on its private dataset $\mathcal{D}_i$. For each local epoch $e = 0, \ldots, E-1$, the model is updated as $\theta_i^{t,e+1} = \theta_i^{t,e} - \eta \nabla F_i(\theta_i^{t,e})$, where η denotes the learning rate. After local training, client $\mathcal{U}_i$ obtains the updated local model $\theta_i^{t,E}$. The corresponding local model update is defined as $\Delta_i^t = \theta_i^{t,E} - \theta^t$.

Aggregation. Each client uploads its local model update Δ_i^t to the server. The server aggregates the received updates to obtain the next global model: $\theta^{t+1} = \theta^t + \sum_{i=1}^{N} p_i \Delta_i^t$.

B. Bottom-ℓ Sketch

Bottom-ℓ sketch [25] is a compact randomized representation that enables efficient estimation of set cardinalities and similarities. Let $\mathcal{I}$ denote a finite universe, and let $h : \mathcal{I} \to (0,1)$ be a hash function whose outputs are modeled as independent and uniformly distributed random variables over $(0,1)$. For a finite set $A \subseteq \mathcal{I}$, the Bottom-ℓ sketch retains the ℓ smallest hash values associated with the elements of A.

Definition 1. *For a finite set $A \subseteq \mathcal{I}$, the Bottom-ℓ sketch of A is defined as $B_\ell(A) = \mathrm{Bottom}_\ell(\{h(a) \mid a \in A\})$, where $\mathrm{Bottom}_\ell(\cdot)$ returns the ℓ smallest values from the input set. If $|A| < \ell$, all hash values are retained.*

A fundamental property of Bottom-ℓ sketches is mergeability [25], which allows the sketch of a set union to be constructed directly from the sketches of individual sets without accessing their original elements.

Lemma 1 (Mergeability of Bottom-ℓ Sketches)**.** *For any two sets $A, B \subseteq \mathcal{I}$, $B_\ell(A \cup B) = \mathrm{Bottom}_\ell(B_\ell(A) \cup B_\ell(B))$.*

This property follows directly from Definition 1. Any hash value discarded from either local sketch cannot belong to the ℓ smallest hash values of the union set. Consequently, the Bottom-ℓ sketch of the union can be constructed solely from the two local sketches, without accessing the original sets.

In addition to mergeability, Bottom-ℓ sketches support cardinality estimation based on uniform order statistics. Let $n = |A|$, and $\tau_A = \max B_\ell(A)$ denote the largest retained hash value, i.e., the ℓ-th smallest value among the n independent uniformly distributed hash values generated by the elements of A. According to the standard result for uniform order statistics, $\tau_A \sim \mathrm{Beta}(\ell, n-\ell+1)$. Specifically, for a random variable $X \sim \mathrm{Beta}(\alpha, \beta)$, its probability density function is

$$f_X(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}, \qquad 0 < x < 1, \tag{1}$$

where $B(\alpha,\beta)$ denotes the beta function. In this setting, $\alpha = \ell$ and $\beta = n - \ell + 1$, because τ_A corresponds to the ℓ-th order statistic among n independent uniform random variables. Using the expectation of the reciprocal of a Beta-distributed random variable, the cardinality of A can be unbiasedly estimated as follows.

Lemma 2 (Cardinality Estimation)**.** *For $n > \ell > 1$, $\mathbb{E}\left[\frac{1}{\tau_A}\right] = \frac{n}{\ell-1}$. Therefore, an unbiased estimator of the cardinality of A is $\widehat{|A|} = (\ell-1)/\tau_A$.*

C. Sparse Polynomial Interpolation

Sparse polynomial interpolation [26] aims to recover a polynomial from a small number of evaluations by exploiting its sparsity rather than its potentially large degree. Consider an s-sparse polynomial $f(x) \in \mathbb{F}_q[x]$ as $f(x) = \sum_{r=1}^{s} c_r x^{e_r}$, where $e_1, \ldots, e_s$ are distinct exponents within a predefined exponent domain $\{0, \ldots, d-1\}$.

The Ben-Or/Tiwari method [27] evaluates $f(x)$ at points forming a geometric sequence. Let $\alpha \in \mathbb{F}_q^*$ be an element whose multiplicative order satisfies $\mathrm{ord}(\alpha) \geq d$, ensuring that α^{e_r} are distinct for different exponents. For each integer $m \geq 0$, define $a_m = f(\alpha^m)$. Substituting the sparse representation of $f(x)$ gives

$$a_m = \sum_{r=1}^{s} c_r (\alpha^m)^{e_r} = \sum_{r=1}^{s} c_r (\alpha^{e_r})^m = \sum_{r=1}^{s} c_r z_r^m, \tag{2}$$

where $z_r = \alpha^{e_r}$. This exponential representation implies that the evaluation sequence satisfies a linear recurrence of order at most s. Specifically, define the locator polynomial

$$\Lambda(z) = \prod_{r=1}^{s} (z - z_r) = \sum_{j=0}^{s} \lambda_j z^j, \tag{3}$$

where $\Lambda(z)$ is monic and $\lambda_s = 1$. Since $\Lambda(z_r) = 0$ for every $r = 1, \ldots, s$ and $m \geq 0$, we have

$$\begin{aligned} \sum_{j=0}^{s} \lambda_j a_{m+j} = \sum_{j=0}^{s} \lambda_j \sum_{r=1}^{s} c_r z_r^{m+j} &= \sum_{r=1}^{s} c_r z_r^m \left(\sum_{j=0}^{s} \lambda_j z_r^j \right) \\ &= \sum_{r=1}^{s} c_r z_r^m \Lambda(z_r) = 0. \end{aligned} \tag{4}$$

Therefore, the locator polynomial can be obtained by solving the above linear recurrence using $2s$ consecutive evaluations $\{a_0, a_1, \ldots, a_{2s-1}\}$. Once $\Lambda(z)$ is recovered, its roots reveal the set $\{z_1, \ldots, z_s\}$, where $z_r = \alpha^{e_r}$. Since the exponent domain is predefined and bounded, the corresponding exponents e_r can be efficiently identified within this domain. Finally, after recovering the exponents, the coefficients $c_1, \ldots, c_s$ are obtained by solving $a_m = \sum_{r=1}^{s} c_r (\alpha^{e_r})^m$.

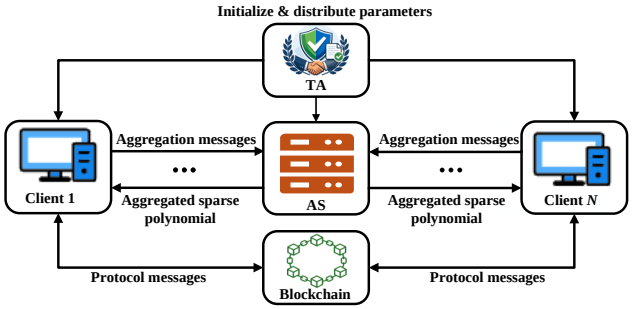


Fig. 1. System model of SPIRIT-FL.

III. Problem Formulation

A. System Model

Fig. 1 illustrates the system model of the proposed scheme, which involves four entities: clients, an aggregation server (AS), a trusted authority (TA), and a blockchain network. Their definitions and interactions are described below:

Clients. The system consists of N clients, denoted by $\mathcal{U} = \{\mathcal{U}_1, \mathcal{U}_2, \ldots, \mathcal{U}_N\}$. Each client $\mathcal{U}_i$ holds a private local dataset

$\mathcal{D}_i$ and performs local training to generate a sparsified model update. Clients upload aggregation messages to the AS, record protocol information on the blockchain, and verify the aggregation result returned by the AS.

AS. The AS coordinates federated training by collecting aggregation messages from clients and recovering the aggregated sparse polynomial. It then broadcasts the aggregated sparse polynomial to all clients for verification.

TA. The TA is responsible for system initialization. It generates the public parameters and secret information required by the protocol and distributes them to the corresponding entities through secure channels. The TA does not participate in the subsequent federated training process.

Blockchain Network. The blockchain provides an immutable and publicly accessible platform for recording protocol information submitted by clients. The recorded information is accessible to all participants to support adaptive weight computation and aggregation verification.

B. Threat Model

Following threat models commonly adopted in recent privacy-preserving and verifiable federated learning schemes [7], [28], [29], we assume that the TA is fully trusted and correctly initializes the system parameters, while the blockchain faithfully provides an immutable and publicly accessible platform for recording protocol-related information. The AS is malicious. Clients are assumed to be honest-but-curious and faithfully execute the prescribed protocol. A subset of clients may additionally collude with the AS by disclosing the information available to them during protocol execution.

Honest-but-curious clients. Honest clients faithfully execute the prescribed protocol, including local training, sparse polynomial encoding, masking, verification, and model updating. However, they may attempt to infer sensitive information about other clients from the information legitimately available to them during protocol execution.

Colluding clients. A subset of clients may collude with the malicious AS by disclosing all protocol information available to them during protocol execution, thereby assisting the AS in inferring sensitive information about non-colluding clients or constructing forged aggregation results. Nevertheless, these colluding clients are assumed to faithfully execute the prescribed protocol and correctly generate all required uploads and verification messages. Therefore, they are protocol-following adversaries rather than Byzantine adversaries.

Participation assumption. We assume that the set of participating clients remains fixed once the secure aggregation procedure begins in each training round, so that pairwise masks can be correctly canceled during aggregation. Clients may join or leave between rounds, while client dropout during an ongoing secure aggregation procedure is outside the scope of this work.

Adversarial models. Depending on whether the AS acts independently or colludes with clients, we consider the following two adversarial models.

- **Weak adversary.** The AS acts independently without colluding with any client, but may deviate from the prescribed aggregation procedure due to laziness, software faults, or malicious intent. Specifically, it may skip part of the aggregation computation or deliberately manipulate the returned aggregated sparse polynomial by omitting, modifying, injecting, or replacing polynomial coefficients. Since the AS does not collude with any client, it cannot obtain the verification points held by the clients during protocol execution.
- **Strong adversary.** The AS colludes with a subset of protocol-following clients. Besides the capabilities of the weak adversary, the colluding parties may jointly exploit the protocol information available to them to infer sensitive information about non-colluding clients from their local sparsified model updates, and carefully construct forged aggregation results intended to evade the proposed verification mechanism. In particular, the AS may obtain the verification points held by the colluding clients, including the common verification point and their own private verification points, but cannot obtain the private verification points of non-colluding clients.

The adversary has the following security objectives.

1) Infer sensitive information about non-colluding clients' local sparsified model updates from the information available during protocol execution.
2) Cause non-colluding clients to accept an incorrect aggregated sparse polynomial during the aggregation verification procedure.

SPIRIT-FL is designed to provide privacy-preserving and verifiable aggregation under protocol-following adversaries. Byzantine behaviors, in which clients arbitrarily deviate from the prescribed protocol by generating malformed protocol messages or incorrect verification messages, are outside the scope of this work.

IV. Design of SPIRIT

A. Overview

SPIRIT-FL follows a round-based workflow that enables privacy-preserving aggregation, trustworthy adaptive weighting, and verifiable aggregation for independently sparsified model updates. Its main stages are summarized as follows.

System initialization. Before federated training begins, the TA initializes the finite-field parameters, pairwise masking seeds, sketch parameters, and verification parameters.

Local sparsified model update generation. In each round, each client performs local training and independently applies Top-k sparsification to obtain a sparsified model update.

Blockchain-based coordinate sketching and adaptive weighting. Each client bucketizes its retained coordinates, constructs a Bottom-ℓ sketch, and uploads the sketch to the blockchain. Based on the public sketches, clients estimate the similarity among bucketized retained coordinate sets and derive consistent aggregation weights without relying on the AS. The merged sketch is further used to estimate the coordinate

coverage for determining the number of evaluations required for sparse polynomial recovery.

Sparse polynomial encoding and aggregation. After the aggregation weights are determined, each client encodes its weighted sparsified model update as a sparse polynomial, evaluates it at public points, and masks the evaluations using pairwise seeds. The AS aggregates the masked evaluations and performs sparse polynomial interpolation to recover the aggregated sparse polynomial without learning individual sparse coordinates or update values.

Aggregation verification and model update. After the AS broadcasts the recovered sparse polynomial, clients verify the aggregation result through public-point and private-point verification. Only if both verification procedures succeed do clients reconstruct the aggregated sparsified model update from the recovered polynomial and update their local models accordingly.

The details of each phase are presented in the following subsections.

B. Initialization Phase

Before the protocol begins, the TA initializes the system parameters as follows.

Finite-field parameters. The TA selects a finite field $\mathbb{F}_q$ and an element $\alpha \in \mathbb{F}_q^*$ whose multiplicative order is at least d, where d denotes the dimension of the flattened model update. The TA also determines a quantization precision parameter Ψ, where the scaling factor 10^{Ψ} is used to convert real-valued model updates into finite-field elements. The value α is published to all clients and the AS. Let $\mathcal{I}_d = \{0, 1, \ldots, d-1\}$ denote the coordinate domain of the flattened model update. The AS precomputes the lookup table $\mathcal{T} = \{\alpha^e \mapsto e \mid e \in \mathcal{I}_d\}$, which enables each recovered field element of the form α^e to be efficiently mapped back to its corresponding coordinate without solving a discrete logarithm.

Pairwise masking seeds. For every pair of clients $\mathcal{U}_i$ and $\mathcal{U}_j$, the TA generates a shared random seed $s_{ij} = s_{ji}$, which is later used to derive pairwise pseudo-random masks for protecting polynomial evaluations during secure aggregation.

Sketch parameters. The TA specifies the Top-k sparsification ratio k, the Bottom-ℓ sketch size ℓ, and the bucket-space size β, where $\beta < d$. Each model coordinate is mapped into the bucket domain $\mathcal{I}_\beta = \{0, 1, \ldots, \beta - 1\}$ through the modulo operation $j \mapsto j \bmod \beta$. The TA further publishes a family of public hash functions $\{h^1, \ldots, h^T\}$, where $h^t : \mathcal{I}_\beta \to (0, 1)$ is generated independently for each communication round. The outputs of each h^t are modeled as independent uniformly distributed random variables over $(0, 1)$.

Verification points. To support aggregation verification, the TA assigns a common verification point set $\{\kappa^t\}_{t=0}^T \in \mathbb{F}_q$, shared by all clients, and a private verification point set $\{\gamma_i^t\}_{t=0}^T \in \mathbb{F}_q$ for each client $\mathcal{U}_i$.

C. Local Training

In each round t $(0 \le t < T)$, each client $\mathcal{U}_i$ initializes its local model from the current global model: $\theta_i^{t,0} = \theta^t$. The client then performs stochastic gradient descent (SGD) over E local epochs on its private dataset $\mathcal{D}_i$: $\theta_i^{t,e+1} = \theta_i^{t,e} - \eta \nabla F_i(\theta_i^{t,e})$, where $F_i(\cdot)$ denotes the local empirical loss function and η is the learning rate and $e = 0, \ldots, E-1$, After local training, client $\mathcal{U}_i$ computes its local model update $\Delta_i^t = \theta_i^{t,E} - \theta_i^{t,0}$. For simplicity, each local update Δ_i^t is flattened into a d-dimensional vector according to a fixed coordinate ordering.

Each client applies Top-k sparsification to its local model update. Given the Top-k sparsification ratio $k \in (0, 1]$, the number of retained coordinates is $K = \lceil kd \rceil$. The retained coordinate set of client $\mathcal{U}_i$ is $\pi_i^t = \mathrm{TopK}(\Delta_i^t, K)$, where $\mathrm{TopK}(\Delta_i^t, K)$ returns the indices of the K entries having the largest absolute values. Hence, $\pi_i^t \subseteq \mathcal{I}_d$, and $|\pi_i^t| = K$. The sparsified model update $\widehat{\Delta}_i^t \in \mathbb{R}^d$ is defined as

$$\widehat{\Delta}_i^t[j] = \begin{cases} \Delta_i^t[j], & j \in \pi_i^t, \\ 0, & j \notin \pi_i^t. \end{cases} \tag{5}$$

where $\Delta_i^t[j]$ denotes the j-th entry of Δ_i^t.

D. Blockchain-based Coordinate Sketching

After obtaining the retained coordinate set π_i^t, each client $\mathcal{U}_i$ constructs a Bottom-ℓ sketch and records it on the blockchain. This stage consists of two steps.

1) Bottom-ℓ Sketch Construction: For each retained coordinate $j \in \pi_i^t$, client $\mathcal{U}_i$ maps it into bucket domain $\mathcal{I}_\beta$ by $b_j = j \bmod \beta$. Then each client $\mathcal{U}_i$ computes the normalized hash value $\nu_j^t = h^t(b_j)$. Let $\mathcal{S}_i^t = \{\nu_j^t \mid j \in \pi_i^t\}$ denote the bucket-hash set constructed from the retained coordinates of client $\mathcal{U}_i$. Since multiple coordinates may be mapped to the same bucket, $\mathcal{S}_i^t$ contains only distinct bucket-level hash values rather than the original retained coordinates.

According to Definition 1, local Bottom-ℓ sketch is

$$\mathcal{B}_i^t = B_\ell(\mathcal{S}_i^t) = \mathrm{Bottom}_\ell(\mathcal{S}_i^t) = \{\nu_{i,1}^t, \nu_{i,2}^t, \ldots, \nu_{i,\ell}^t\}. \tag{6}$$

where $\nu_{i,1}^t < \nu_{i,2}^t < \cdots < \nu_{i,\ell}^t$. Finally, client $\mathcal{U}_i$ uploads its local Bottom-ℓ sketch $\mathcal{B}_i^t$ to the blockchain.

2) Estimated Jaccard Similarity and Weight Computation: After all clients publish their local Bottom-ℓ sketches to the blockchain, every client retrieves the public Bottom-ℓ sketches and independently estimates the pairwise Jaccard similarities, from which the aggregation weights are subsequently derived.

For two clients $\mathcal{U}_i$ and $\mathcal{U}_j$, the estimated Jaccard similarity between their bucketized retained coordinate sets is computed from the two Bottom-ℓ sketches as $\widehat{J}_{i,j}^t = |\mathcal{B}_i^t \cap \mathcal{B}_j^t| / |\mathcal{B}_i^t \cup \mathcal{B}_j^t|$.

Since the number of retained coordinates after Top-k sparsification is approximately kd, and $kd \gg \ell$ in our setting, every Bottom-ℓ sketch contains exactly ℓ hash values. Hence, $|\mathcal{B}_i^t| = |\mathcal{B}_j^t| = \ell$. By the inclusion–exclusion principle, the union cardinality is

$$|\mathcal{B}_i^t \cup \mathcal{B}_j^t| = |\mathcal{B}_i^t| + |\mathcal{B}_j^t| - |\mathcal{B}_i^t \cap \mathcal{B}_j^t| = 2\ell - |\mathcal{B}_i^t \cap \mathcal{B}_j^t|, \tag{7}$$

which yields the equivalent expression

$$\widehat{J}_{i,j}^t = \frac{|\mathcal{B}_i^t \cap \mathcal{B}_j^t|}{2\ell - |\mathcal{B}_i^t \cap \mathcal{B}_j^t|}. \tag{8}$$

A larger estimated Jaccard similarity indicates a higher overlap between the bucketized retained coordinate sets of the two clients. The coordinate consensus score of client $\mathcal{U}_i$ is computed as $\sigma_i^t = \frac{1}{N-1}\sum_{j=1,\, j\neq i}^{N} \widehat{J}_{i,j}^t$. Since all clients use the same public Bottom-ℓ sketches, they obtain the same consensus score vector $(\sigma_1^t, \sigma_2^t, \ldots, \sigma_N^t)$. The aggregation weight is then obtained through the softmax normalization

$$w_i^t = \frac{\exp(\sigma_i^t)}{\sum_{j=1}^{N}\exp(\sigma_j^t)}. \tag{9}$$

Clients whose retained coordinate sets exhibit higher consensus with the majority receive larger aggregation weights.

E. Verification-Oriented Coverage Estimation

Besides aggregation weighting, the public Bottom-ℓ sketches are also used to estimate the bucket-level support coverage for determining the sparse polynomial interpolation budget. Since the sketch is constructed over the bucket domain, the obtained estimate corresponds to the number of occupied buckets rather than the exact coordinate-level support size.

According to the recursive mergeability property of Bottom-ℓ sketches (Lemma 1), the global Bottom-ℓ sketch can be constructed by recursively merging all local sketches:

$$\mathcal{B}_{\text{glob}}^t = B_\ell\left(\bigcup_{i=1}^{N}\mathcal{S}_i^t\right) = \text{Bottom}_\ell\left(\bigcup_{i=1}^{N}\mathcal{B}_i^t\right), \tag{10}$$

where $\mathcal{B}_{\text{glob}}^t = \{\nu_1^t, \nu_2^t, \ldots, \nu_\ell^t\}$, with $\nu_1^t < \nu_2^t < \cdots < \nu_\ell^t$.

Let $\tau_\ell^t = \nu_\ell^t$ denote the largest retained hash value in the merged sketch. According to Lemma 2, the bucket-level support coverage is estimated as $\widehat{S}^t = (\ell-1)/\tau_\ell^t$. Equivalently, $\widehat{S}^t \approx \left|\bigcup_{i=1}^{N}\{j \bmod \beta \mid j \in \pi_i^t\}\right|$, which estimates the number of distinct buckets occupied by the union of all clients' retained coordinate sets after bucketization.

The interpolation budget is further enlarged by a redundancy term to compensate for Bottom-ℓ estimation uncertainty and the gap between bucket-level and coordinate-level supports.

F. Aggregation and Update

1) Polynomial Encoding: After obtaining the sparsified model update $\hat{\Delta}_i^t$ and the aggregation weight w_i^t, client $\mathcal{U}_i$ encodes the weighted sparsified model update into a sparse polynomial over the finite field $\mathbb{F}_q$. Specifically, each retained coordinate $j \in \pi_i^t$ is used as the polynomial exponent, while the corresponding weighted sparsified model update $w_i^t\hat{\Delta}_i^t[j]$ is quantized and mapped into $\mathbb{F}_q$. Client $\mathcal{U}_i$ first quantizes the weighted sparsified model update value as $\bar{\Delta}_i^t[j] = \left\lfloor w_i^t\hat{\Delta}_i^t[j]\cdot 10^{\Psi}\right\rfloor$, where Ψ is the quantization precision introduced during initialization. The resulting value $\bar{\Delta}_i^t[j] \in \mathbb{Z}$ is the quantized integer representation of the weighted sparsified model update at coordinate j. The corresponding finite-field element is $\widetilde{\Delta}_i^t[j] = \bar{\Delta}_i^t[j] \bmod q$. Accordingly, client $\mathcal{U}_i$ encodes its weighted sparsified model update as the sparse polynomial

$$f_i(x) = \sum_{j\in\pi_i^t}\widetilde{\Delta}_i^t[j]x^j. \tag{11}$$

2) Commitment Generation: Given the local sparse polynomial $f_i(x)$, client $\mathcal{U}_i$ evaluates $f_i(x)$ at its private verification point γ_i^t to obtain the evaluation $f_i(\gamma_i^t)$. The client then computes the commitment $\mathsf{Com}_i^t = \mathsf{Hash}(\gamma_i^t \| f_i(\gamma_i^t))$, where $\mathsf{Hash}(\cdot)$ is a cryptographic hash function and $\|$ denotes concatenation. Each client $\mathcal{U}_i$ publishes the commitment Com_i^t to the blockchain while keeping γ_i^t and $f_i(\gamma_i^t)$ undisclosed.

3) Evaluation Point Uploading: Based on the estimated bucket-level support coverage $\widehat{S}^t$, each client determines the number of evaluation points as $2M = 2\omega(\widehat{S}^t + \delta^t)$, where δ^t provides a statistical margin derived from the estimation uncertainty of the Bottom-ℓ cardinality estimator, and $\omega > 1$ is a safety expansion factor that compensates for the possible discrepancy between the estimated bucket-level support coverage and the actual coordinate-level support size. Since the relative standard error of the Bottom-ℓ estimator is approximately $1/\sqrt{\ell-2}$, we set

$$\delta^t = \left\lceil \frac{\widehat{S}^t}{\sqrt{\ell-2}} \right\rceil. \tag{12}$$

The factor ω enlarges the evaluation budget to tolerate possible underestimation caused by bucket collisions during sketch construction.

Then, client $\mathcal{U}_i$ evaluates $f_i(x)$ at the public evaluation points $\{\alpha^m\}_{m=0}^{2M-1}$. The local evaluation set of client $\mathcal{U}_i$ is

$$\mathcal{E}_i^t = \{(\alpha^m, f_i(\alpha^m)) \mid m \in \{0,1,\ldots,2M-1\}\}. \tag{13}$$

To protect evaluation privacy, client $\mathcal{U}_i$ masks each evaluation using the pairwise mask

$$\xi_i^m = \sum_{\mathcal{U}_j\in\mathcal{U},\, i<j}\text{PRG}\left(s_{ij}\|m\|t\right) - \sum_{\mathcal{U}_j\in\mathcal{U},\, i>j}\text{PRG}\left(s_{ij}\|m\|t\right), \tag{14}$$

where $\text{PRG}(\cdot)$ denotes a pseudo-random generator. The masked evaluation is $\widetilde{f}_i(\alpha^m) = f_i(\alpha^m) + \xi_i^m$.

Finally, each client $\mathcal{U}_i$ uploads the masked evaluation set $\widetilde{\mathcal{E}}_i^t = \left\{(\alpha^m, \widetilde{f}_i(\alpha^m)) \mid m = 0,\ldots,2M-1\right\}$ to AS.

4) Aggregated Sparse Polynomial Recovery: After collecting the masked evaluation sets from all clients, the AS recovers the aggregated sparse polynomial using the Ben-Or/Tiwari sparse polynomial interpolation algorithm.

Determining the Number of Nonzero Terms. For every public evaluation point α^m, the AS aggregates the received masked evaluations. Since the pairwise masks satisfy $\sum_{i=1}^{N}\xi_i^m = 0$, all masks cancel after summation. Therefore, the aggregated evaluation is

$$Y_m \triangleq \mathcal{F}(\alpha^m) = \sum_{i=1}^{N} f_i(\alpha^m). \tag{15}$$

Let S denote the number of nonzero terms in $\mathcal{F}(x)$, and define $z_r = \alpha^{e_r}$ $(r = 1,\ldots,S)$. Using Eq. 2, the aggregated evaluation sequence can be written as $Y_m = \sum_{r=1}^{S} c_r z_r^m$. To determine the sparsity of the aggregated polynomial, the AS constructs the Hankel matrix

$$H_0 = \begin{bmatrix} Y_0 & Y_1 & \cdots & Y_{M-1} \\ Y_1 & Y_2 & \cdots & Y_M \\ \vdots & \vdots & \ddots & \vdots \\ Y_{M-1} & Y_M & \cdots & Y_{2M-2} \end{bmatrix}, \tag{16}$$

The sparsity is then obtained as $S = \mathrm{rank}(H_0)$.

Recovering the Locator Polynomial. Using the linear recurrence in Eq. 4, the sequence $\{Y_0, \ldots, Y_{2S-1}\}$ satisfies the characteristic (locator) polynomial

$$\Lambda(z) = \prod_{r=1}^{S}(z - z_r) = z^S + \lambda_{S-1}z^{S-1} + \cdots + \lambda_1 z + \lambda_0. \tag{17}$$

The locator coefficients $\lambda_0, \ldots, \lambda_{S-1}$ are obtained by solving the Hankel linear system

$$\begin{bmatrix} Y_0 & Y_1 & \cdots & Y_{S-1} \\ Y_1 & Y_2 & \cdots & Y_S \\ \vdots & \vdots & \ddots & \vdots \\ Y_{S-1} & Y_S & \cdots & Y_{2S-2} \end{bmatrix} \begin{bmatrix} \lambda_0 \\ \lambda_1 \\ \vdots \\ \lambda_{S-1} \end{bmatrix} = - \begin{bmatrix} Y_S \\ Y_{S+1} \\ \vdots \\ Y_{2S-1} \end{bmatrix}. \tag{18}$$

After recovering $\Lambda(z)$, AS factors it over $\mathbb{F}_q$ using the randomized Cantor–Zassenhaus algorithm to obtain the root set $\mathcal{Z} = \{z_1, \ldots, z_S\}$. Further details are provided in **Appendix A**.

Recovering the Coordinate Indices. For every recovered root $z_r = \alpha^{e_r}$ $(r \in 1, 2, ..., S)$, the AS searches the lookup table $\mathcal{T}$ generated during initialization and obtains the corresponding exponent $e_r = \mathcal{T}(z_r)$. The AS obtains the recovered coordinate set $\{e_1, e_2, \ldots, e_S\}$.

Recovering the Polynomial Coefficients. After recovering the exponents, the AS recovers the coefficient set $\{c_1, c_2, \ldots, c_S\}$. Since $z_r = \alpha^{e_r}$, the coefficients are obtained by solving the following Vandermonde system:

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ z_1 & z_2 & \cdots & z_S \\ z_1^2 & z_2^2 & \cdots & z_S^2 \\ \vdots & \vdots & \ddots & \vdots \\ z_1^{S-1} & z_2^{S-1} & \cdots & z_S^{S-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_S \end{bmatrix} = \begin{bmatrix} Y_0 \\ Y_1 \\ \vdots \\ Y_{S-1} \end{bmatrix}. \tag{19}$$

Using the recovered exponents and coefficients, the AS reconstructs the aggregated sparse polynomial $\mathcal{F}(x) = \sum_{r=1}^{S} c_r x^{e_r}$. Equivalently, its sparsified coordinate–coefficient representation $\{(e_r, c_r)\}_{r=1}^{S}$, where each pair (e_r, c_r) consists of one recovered coordinate and its corresponding aggregated sparsified model update value.

Finally, the AS broadcasts the recovered aggregated sparse polynomial $\mathcal{F}(x)$ to all clients.

G. Aggregation Verification

After receiving the aggregated sparse polynomial $\mathcal{F}(x)$ broadcast by the AS, each client first checks the validity of the recovered exponents. Specifically, for every recovered coordinate–coefficient pair (e_r, c_r) in $\mathcal{F}(x)$, the client verifies whether $e_r \in \mathcal{I}_d = \{0, \ldots, d-1\}$. If any exponent falls outside the valid coordinate domain, the aggregation result is rejected. After the exponent validity check, each client verifies whether the returned aggregation result is correct. The verification consists of public-point verification and private-point verification.

1) Public-point Verification: Each client $\mathcal{U}_i$ first evaluates its sparse polynomial $f_i(x)$ at the public verification point κ^t to obtain the evaluation $f_i(\kappa^t)$. Then, each client publishes the evaluation $f_i(\kappa^t)$, and all clients perform public-point verification. The public-point verification is successful if the following equality holds:

$$\mathcal{F}(\kappa^t) \stackrel{?}{=} \sum_{i=1}^{N} f_i(\kappa^t). \tag{20}$$

All clients can independently evaluate $\mathcal{F}(\kappa^t)$ and verify the above equality. If the equality holds, the aggregated sparse polynomial is consistent with the publicly released evaluations.

2) Private-point Verification: If the public-point verification succeeds, the clients proceed to private-point verification. Each client $\mathcal{U}_i$ reveals its private verification point γ_i^t to the other clients. For each client $\mathcal{U}_i$, every other client $\mathcal{U}_j$ $(j \neq i)$ evaluates its own sparse polynomial at γ_i^t and broadcasts the corresponding evaluation $f_j(\gamma_i^t)$ to all clients. Only after all other clients have uploaded their evaluations at γ_i^t does $\mathcal{U}_i$ reveal the evaluation $f_i(\gamma_i^t)$ on the blockchain. For each client $\mathcal{U}_i$, the other clients verify that the revealed verification point and evaluation match the previously published commitment by checking $\mathsf{Com}_i^t \stackrel{?}{=} \mathsf{Hash}(\gamma_i^t \| f_i(\gamma_i^t))$.

After the commitment verification succeeds, all clients verify

$$\mathcal{F}(\gamma_i^t) \stackrel{?}{=} \sum_{j=1}^{N} f_j(\gamma_i^t), \qquad i = 1, \ldots, N. \tag{21}$$

If the equality holds for every private verification point, the private-point verification succeeds.

3) Verification Decision and Repeated Recovery: The verification result is determined as follows:

- **Rule 1:** If both **Eq. 20** and **Eq. 21** hold, the aggregated sparse polynomial $\mathcal{F}(x)$ is accepted as correct.
- **Rule 2:** If **Eq. 20** holds but **Eq. 21** fails, the AS is regarded as a strong attacker, and $\mathcal{F}(x)$ is rejected as a forged aggregation result.
- **Rule 3:** If **Eq. 20** fails, the failure may result from either a malicious AS or insufficient evaluation points for sparse polynomial recovery. The clients initiate a new recovery–verification attempt within the current FL round, without repeating local training or sparse polynomial encoding. The clients upload $2\rho M$ newly masked polynomial evaluations, where $\rho > 1$ is the recovery expansion factor. The AS reconstructs and broadcasts the aggregated sparse polynomial again, and the clients reperform verification using fresh evaluation points. If the public-point verification still fails after recovery, the AS is regarded as a weak attacker and the returned polynomial is rejected.

If the returned aggregation polynomial passes the above verification procedure, each client accepts the aggregation result and proceeds to local model update.

H. Client Local Update

After the aggregated sparse polynomial $\mathcal{F}(x)$ is successfully verified, each client reconstructs the aggregated sparsified model update by converting the recovered polynomial coefficients from $\mathbb{F}_q$ back to the real domain, and then updates its local model accordingly.

For each recovered coordinate–coefficient pair (e_r, c_r) contained in $\mathcal{F}(x)$, the client first converts the finite-field coefficient into a signed integer:

$$\mathrm{Signed}(c_r) = \begin{cases} c_r, & c_r \le q/2, \\ c_r - q, & c_r > q/2. \end{cases} \tag{22}$$

The corresponding real-valued sparsified model update is then recovered as $\bar{c}_r = \mathrm{Signed}(c_r)/10^{\Psi}$, where Ψ is the quantization precision introduced during initialization.

Next, each client reconstructs the aggregated sparsified model update $g^t_{\mathrm{agg}} \in \mathbb{R}^d$. The vector is initialized as an all-zero vector. For every recovered coordinate–coefficient pair (e_r, c_r), the corresponding entry is assigned as

$$\Delta^t_{\mathrm{agg}}[j] = \begin{cases} \bar{c}_r, & j = e_r,\ r = 1, \dots, S, \\ 0, & \text{otherwise.} \end{cases} \tag{23}$$

Finally, each client updates its local model according to

$$\theta^{t+1} = \theta^t + \Delta^t_{\mathrm{agg}}. \tag{24}$$

V. THEORETICAL ANALYSIS

A. Privacy Analysis

We analyze the privacy of SPIRIT-FL against masked evaluation leakage, sparse coordinate leakage, verification leakage, and collusion-induced leakage through the following theorems.

Theorem 1. *Assume that the AS colludes with at most $N-2$ clients and that the PRG is secure. The masked evaluation set uploaded by an honest client leaks no information about its local sparse polynomial evaluations beyond the aggregate evaluations of honest clients. Specifically, any two sets of honest-client polynomial evaluations with identical aggregate evaluations induce computationally indistinguishable adversarial views.*

The proof of Theorem 1 can be found in Appendix B-A1.

The sparse coordinate support may also leak information about clients' local data. Since gradient coordinates are data-dependent, the exact Top-k support reflects the locations of dominant gradient components and can serve as structural constraints for gradient inversion attacks [30], [31], [32]. In contrast, the Bottom-ℓ sketch exposes only sampled bucket-level support, where each bucket corresponds to multiple possible coordinates, making the exact Top-k pattern difficult to recover from the sketch alone. We further evaluate this coordinate leakage in Appendix C-B.

Theorem 2. *The Bottom-ℓ sketch exposes sampled bucket-level support information and approximate inter-client overlap, but does not uniquely determine the exact Top-k coordinate set within a single communication round.*

The proof of Theorem 2 can be found in Appendix B-A2.

Theorem 3. *Let $f_i(x)$ be the sparse polynomial of an honest client $\mathcal{U}_i$, and let $\mathcal{S}_i = \mathrm{supp}(f_i)$ denote its support, with $s_i = |\mathcal{S}_i|$. Let Γ_i denote the set of verification points at which evaluations of $f_i(x)$ are disclosed during verification. If $|\Gamma_i| < s_i$ and $q - 1 > s_i$, then the disclosed verification evaluations do not uniquely determine $f_i(x)$ over $\mathbb{F}_q$, even if the adversary is additionally given the exact support $\mathcal{S}_i$. Consequently, the disclosed verification evaluations alone are insufficient to uniquely reconstruct the weighted sparsified model update.*

The proof of Theorem 3 can be found in Appendix B-A3.

Theorem 4 (Joint-View Privacy under Collusion)**.** *Assuming that the PRG is secure, the joint view of a malicious AS colluding with a subset of protocol-following clients can be computationally simulated from the colluding clients' protocol states, the aggregated sparse polynomial $F^t_{\mathcal{H}}(x)$ of the non-colluding clients, the public protocol parameters, and the explicitly characterized leakage revealed by the sketching and verification procedures. Therefore, conditioned on this leakage, the protocol transcript reveals no additional information about the individual sparse polynomials of non-colluding clients beyond the explicitly characterized leakage.*

The proof of Theorem 4 can be found in Appendix B-A4.

B. Correctness Analysis

Theorem 5. *Assume that all clients, the TA, and the AS honestly execute SPIRIT-FL. If the initial evaluation budget satisfies $2M \ge 2S$, or the initial recovery fails verification and the repeated evaluation-point uploading procedure provides $2\rho M \ge 2S$ evaluations, the multiplicative order of α is at least d, and the field size q is sufficiently large to avoid modular wrap-around of the aggregated quantized coefficients, then the AS correctly recovers the aggregated sparse polynomial $\mathcal{F}(x)$, and all honest clients accept the aggregation result through the two-stage aggregation verification.*

The proof of Theorem 5 can be found in Appendix B-B.

C. Verifiability Analysis

We define verifiability as the ability of non-colluding clients to detect an incorrect aggregated sparse polynomial returned by the AS. Let $\mathcal{F}(x)$ denote the correct aggregated sparse polynomial. An incorrect aggregation returned by the AS is represented as $\widehat{\mathcal{F}}(x) = \mathcal{F}(x) + \Phi(x)$, where $\Phi(x) \neq 0$ is referred to as the deviation polynomial. According to the two adversarial models defined in Section III-B, we establish the following two verifiability theorems.

Theorem 6. *Under the weak attack model, if the AS returns an incorrect aggregated sparse polynomial, the public-point*

verification detects the deviation except with probability at most $(d-1)/q$.

The proof of Theorem 6 can be found in Appendix B-C1.

Theorem 7. *Under the strong attack model, if the AS returns an incorrect aggregated sparse polynomial that passes the public-point verification, the private-point verification detects the incorrect aggregated sparse polynomial except with probability at most* $(d-1)/q$.

The proof of Theorem 7 can be found in Appendix B-C2.

VI. EVALUATION

A. Experimental Setup

1) Datasets and Models: To evaluate the performance of SPIRIT-FL, we conduct experiments on three benchmark datasets: MNIST [33], Fashion-MNIST [34], and CIFAR-10 [35]. For each dataset, experiments are conducted under both IID and Non-IID settings. Under the IID setting, the training samples are randomly and approximately uniformly partitioned across participating clients. Under the Non-IID setting, the data partition among clients is generated using a Dirichlet distribution [36] with concentration parameter $\alpha = 1$.

For MNIST and Fashion-MNIST, we employ a convolutional neural network (CNN), whereas ResNet-10 [37] is used for CIFAR-10. Unless otherwise specified, we use $N = 20$ participating clients, 5 local epochs per communication round, a learning rate of 0.1, a local mini-batch size of 64, and a Top-k sparsification ratio of 0.25%. Additional details of the datasets, model architectures, and training configurations are provided in Appendix C-A.

2) System Configuration: The federated learning experiments are conducted using PyTorch [38] on a workstation equipped with an Intel Core i7-11700K CPU (8 cores, 3.60 GHz), 64 GB RAM, and Ubuntu 22.04.5 LTS. The blockchain component is implemented in Solidity 0.8.28 and evaluated on a local Hardhat network. Smart contracts are deployed and evaluated using TypeScript scripts based on viem 2.55.2.

B. Efficiency Evaluation

1) Baseline Schemes: To evaluate the efficiency of SPIRIT-FL, we compare it with the following four representative privacy-preserving and verifiable federated learning schemes [39], [40], [41], [42] in terms of communication overhead, computation overhead, and blockchain operation overhead.

- **VCD-FL [39].** VCD-FL protects local model updates through lightweight masking and verifies aggregation correctness using polynomial-based commitments.
- **SWAS [40].** SWAS protects local model updates through pseudorandom masking and verifies aggregation correctness using compact polynomial evaluations.
- **VLMR-SFL [41].** VLMR-SFL protects local model updates through secure aggregation and verifies aggregation correctness using homomorphic hashing.

TABLE I
COMMUNICATION OVERHEAD COMPARISON OF PRIVACY-PRESERVING AND VERIFIABLE FL SCHEMES.

Scheme	M'	**CNN**		**ResNet-10**
		MNIST	**Fashion-MNIST**	**CIFAR-10**
# of parameters	–	582,026	582,026	4,910,922
Plaintext size	–	2.220 MB	2.220 MB	18.734 MB
VCD-FL / SWAS	64	2.256 MB	2.256 MB	19.031 MB
VCD-FL / SWAS	128	2.238 MB	2.238 MB	18.881 MB
VLMR-SFL	–	2.254 MB	2.254 MB	18.768 MB
DSVFL	–	0.356 MB	0.356 MB	2.998 MB
SPIRIT-FL (32-bit)	–	**0.019 MB**	**0.018 MB**	**0.118 MB**
SPIRIT-FL (128-bit)	–	**0.072 MB**	**0.069 MB**	**0.468 MB**

Note: The communication overhead denotes the transmitted information from one client in one communication round. Model parameters are represented using 32 bits. For SPIRIT-FL, the reported overhead includes the polynomial evaluations used for sparse recovery, the Bottom-ℓ sketch, the commitment, and the information disclosed during aggregation verification. Polynomial evaluations over $\mathbb{F}_q$ are represented using either 32-bit or 128-bit finite-field elements. For DSVFL, the ciphertext size is calculated according to the Paillier encryption setting with a 1024-bit security parameter.

- **DSVFL [42].** DSVFL supports Top-k gradient sparsification and protects retained gradient values through Paillier-based secure aggregation. It processes the corresponding coordinate information for sparse aggregation and verifies aggregation correctness using client signatures supported by Shamir secret sharing.

2) Communication Overhead: We compare the communication overhead of different privacy-preserving and verifiable FL schemes under the CNN and ResNet-10 models. The detailed communication overhead derivation and parameter settings are provided in Appendix C-C1. As shown in Table I, VCD-FL, SWAS, and VLMR-SFL incur communication overheads close to the plaintext model size because their verification mechanisms require transmitting full-dimensional model updates or encrypted model values. DSVFL reduces communication through Top-k sparsification, but each retained gradient must still be individually encrypted using Paillier, resulting in considerable ciphertext expansion.

In contrast, SPIRIT-FL transmits sparse polynomial evaluations together with a Bottom-ℓ sketch, a commitment, and verification information. We evaluate SPIRIT-FL using both 32-bit and 128-bit finite-field elements. For $N = 20$, the reported per-client communication overhead also accounts for the $N+2$ finite-field elements transmitted during verification. The 32-bit setting is included only to illustrate the communication cost under a smaller field, whereas the theoretical security analysis and the default secure configuration use the 128-bit field setting. Even under the 128-bit setting, SPIRIT-FL incurs only 0.468 MB on CIFAR-10 with ResNet-10, representing a 6.4-fold reduction compared with DSVFL and up to approximately 40-fold reductions compared with the other baseline schemes. Thus, the communication overhead of SPIRIT-FL mainly depends on the number of polynomial evaluations required for sparse recovery rather than the full model dimension.

3) Computation Overhead: Fig. 2 compares the computation overhead of SPIRIT-FL and the baseline schemes in the encoding/encryption, aggregation, decoding/decryption, and

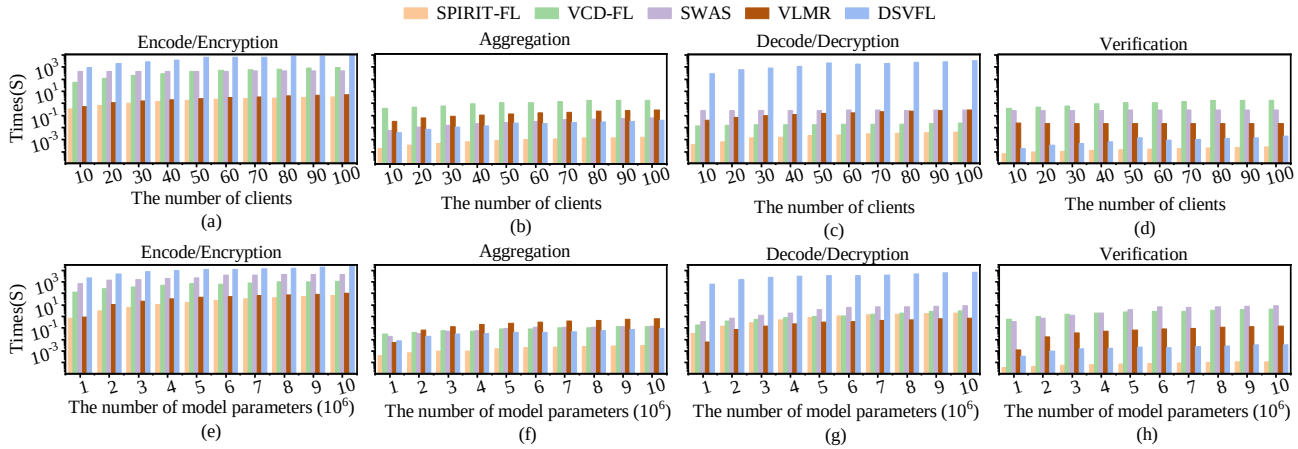


Fig. 2. Computation overhead of SPIRIT-FL and the baseline schemes under different numbers of participating clients with a fixed model dimension ($d = 10^6$) ((a)–(d)), and under different model dimensions with a fixed number of participating clients ($N = 20$) ((e)–(h)).

TABLE II
BLOCKCHAIN OPERATION OVERHEAD PER CLIENT IN SPIRIT-FL.

Operation	Gas	Calldata (B)*	Transaction latency (ms)
Bottom-ℓ sketch submission	$71{,}655 \pm 9$	1,156	72.481 ± 11.541
Commitment submission	68,869	100	67.666 ± 9.871
Verification-value disclosure	32,202	132	70.711 ± 8.351
Total per client per communication round	$172{,}726 \pm 9$	1,388	210.858 ± 14.755

Setting. $d = 582{,}026$, $k = 0.0025$, $kd = 1{,}456$, $\beta = 10{,}000$, $\ell = 128$, and 64-bit hash values.

* The reported calldata includes both the transaction payload and the ABI metadata, including a 4-byte function selector, a 32-byte round identifier, and a 32-byte client identifier.

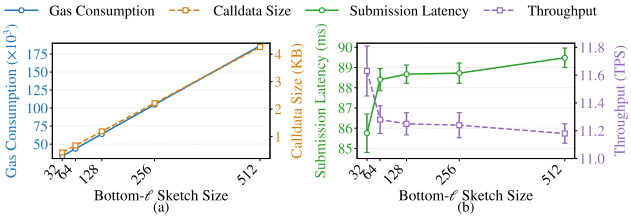


Fig. 3. Impact of the Bottom-ℓ sketch size on blockchain performance. (a) Gas consumption and calldata size. (b) Transaction latency and throughput.

verification phases. For Fig. 2(a)–(d), the model dimension is fixed at $d = 10^6$, while Fig. 2(e)–(h) vary the model dimension with $N = 20$ participating clients. Overall, the computation overhead of all schemes increases with either the number of participating clients or the model dimension. SPIRIT-FL incurs lower computation overhead than the baseline schemes in most settings. The advantage is particularly pronounced in the aggregation and verification phases, where SPIRIT-FL achieves reductions of several orders of magnitude over multiple baseline schemes in the evaluated settings. The reduction mainly comes from combining Top-k sparsification with sparse polynomial encoding and recovery, which avoids applying expensive cryptographic operations to full-dimensional model updates. In contrast, VCD-FL and SWAS additionally involve polynomial-interpolation-based verification operations, VLMR relies on homomorphic hashing and bilinear pairing operations, and DSVFL requires Paillier encryption and decryption. DSVFL incurs particularly high encoding/encryption and decoding/decryption costs due to its Paillier-based encryption and decryption operations.

4) Blockchain Operation Overhead: We evaluate the overhead of the three on-chain operations in SPIRIT-FL. As shown in Table II, Bottom-ℓ sketch submission incurs the largest communication and gas overhead because it uploads the complete sketch, whereas the other two operations transmit only fixed-size cryptographic values (Com_i^t and $f_i(\gamma_i^t)$). Overall, each client incurs approximately 172,726 gas, 1,388 bytes of calldata, and 210 ms of round completion latency per communication round. The results show that SPIRIT-FL only requires storing compact sketches, commitments, and verification values on the blockchain, without storing high-dimensional model updates or sparsified model updates.

C. Effectiveness Analysis

Unless otherwise specified, all experiments in this subsection are conducted on Fashion-MNIST under the Non-IID setting using the CNN model.

1) Impact of the Bottom-ℓ Sketch Size: We evaluate the impact of the Bottom-ℓ sketch size on blockchain overhead and estimation accuracy. The number of participating clients is fixed at 20, while the sketch size varies over $\ell \in \{32, 64, 128, 256, 512\}$. As shown in Fig. 3(a), both gas consumption and calldata size increase approximately linearly with the Bottom-ℓ sketch size because larger Bottom-ℓ sketches require more on-chain storage and transmission. Specifically, the average gas consumption increases from approximately 3.3×10^4 at $\ell = 32$ to 1.9×10^5 at $\ell = 512$, while the calldata size increases from 420 bytes to 4,260 bytes.

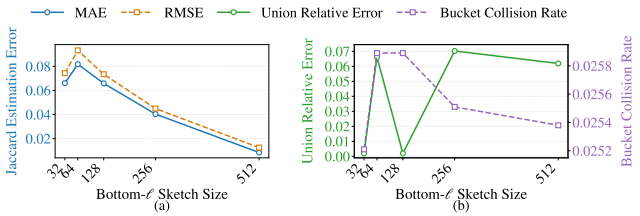


Fig. 4. Impact of the Bottom-ℓ sketch size on estimation accuracy. (a) Pairwise Jaccard estimation error. (b) Union-size relative error and bucket collision ratio.

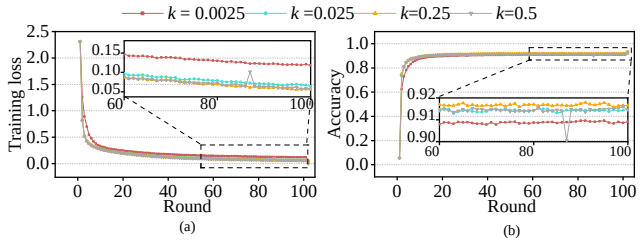


Fig. 5. Impact of the Top-k sparsification ratio on model utility. (a) Training loss. (b) Test accuracy.

In contrast, Fig. 3(b) shows that transaction latency remains within 84.7–87.5 ms and the throughput remains around 11.4–11.8 TPS as ℓ increases, showing that increasing the sketch size has little influence on transaction latency and throughput.

To evaluate estimation accuracy, let J_{ij} and $\widehat{J}_{ij}$ denote the true and estimated Jaccard similarities between clients $\mathcal{U}_i$ and $\mathcal{U}_j$, respectively. The mean absolute error (MAE) and root mean squared error (RMSE) are computed over all client pairs as $\mathrm{MAE} = \frac{1}{|\mathcal{U}|}\sum_{(i,j)\in\mathcal{U}}\left|\widehat{J}_{ij} - J_{ij}\right|$ and $\mathrm{RMSE} = \sqrt{\frac{1}{|\mathcal{U}|}\sum_{(i,j)\in\mathcal{P}}\left(\widehat{J}_{ij} - J_{ij}\right)^2}$ where $\mathcal{P}$ denotes the set of all client pairs. Figure 4(a) shows that both MAE and RMSE decrease as ℓ increases, indicating improved Jaccard estimation accuracy with larger sketches. Let U and $\widehat{U}$ denote the true and estimated coordinate union sizes, respectively. The union-size relative error is defined as $\mathrm{RE}_U = \left|\widehat{U} - U\right|/U$. Before constructing the sketch, coordinates are mapped into β buckets. The bucket collision ratio is defined as $\mathrm{CR}_{\mathrm{bucket}} = (U_{\mathrm{coord}} - U_{\mathrm{bucket}})/U_{\mathrm{coord}}$, where U_{coord} and U_{bucket} denote the coordinate union sizes before and after bucket mapping, respectively. As shown in Fig. 4(b), union-size relative error remains low for all evaluated sketch sizes, while the bucket collision ratio remains nearly unchanged because it is determined mainly by the bucket size β. Therefore, increasing the Bottom-ℓ sketch size improves estimation accuracy but also increases blockchain overhead. Considering that $\ell = 128$ provides low estimation error while incurring relatively low blockchain overhead, we adopt $\ell = 128$ in the remaining experiments.

2) Impact of the Top-k Sparsification Ratio: To evaluate the impact of the Top-k sparsification ratio on model utility, we vary $k \in \{0.0025, 0.025, 0.25, 0.5\}$ while keeping all other settings unchanged. Fig. 5 shows the training loss and test accuracy over 100 communication rounds.

As shown in Fig. 5(a), the training loss generally decreases as the Top-k sparsification ratio increases because larger Top-k ratios retain more significant update components in each communication round. Fig. 5(b) shows that increasing the Top-k sparsification ratio does not monotonically improve the final model accuracy. Among the evaluated settings, $k = 0.25$ achieves the highest accuracy of approximately 91.6%. Even with the most aggressive Top-k sparsification ratio, $k = 0.0025$, where each client retains only 0.25% of the model parameters, SPIRIT-FL still achieves an accuracy of approximately 90.8%, which is less than one percentage point lower than the best-performing setting. These results demonstrate that SPIRIT-FL maintains stable model utility across different sparsification ratios. Considering the tradeoff between communication efficiency and model utility, we use $k = 0.0025$ in the remaining experiments.

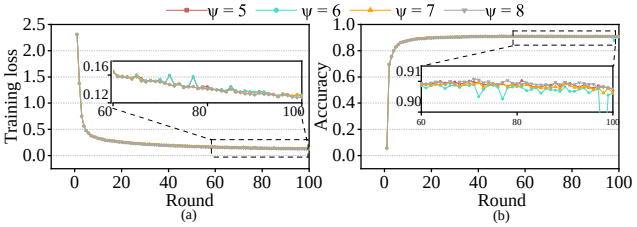


Fig. 6. Impact of the quantization precision Ψ on model utility. (a) Training loss. (b) Test accuracy.

3) Impact of the Quantization Precision: To evaluate the impact of the quantization precision Ψ, we vary $\Psi \in \{5, 6, 7, 8\}$ while keeping all other settings unchanged. Fig. 6 shows the training loss and test accuracy over 100 communication rounds.

As shown in Fig. 6(a), increasing the quantization precision slightly reduces the final training loss. However, the improvement becomes marginal once $\Psi \geq 6$, indicating that further increasing the precision brings limited benefits. Fig. 6(b) shows that all evaluated settings achieve nearly identical test accuracies throughout the training process, with the final accuracy remaining around $90\% - -91\%$. These results indicate that SPIRIT-FL is insensitive to the quantization precision within the evaluated range. Therefore, we set $\Psi = 6$ in the remaining experiments because it achieves comparable model utility with the smallest evaluated quantization precision.

4) Effectiveness of the Sketch-Based Similarity Metric: To evaluate the effectiveness of the sketch-based similarity metric, we conduct experiments on MNIST with seven users $\{\mathcal{U}_1, \ldots, \mathcal{U}_7\}$. User $\mathcal{U}_1$ contains samples from labels 0–4, whereas users $\mathcal{U}_2$–$\mathcal{U}_7$ are constructed to have label-overlap ratios of $0\%, 20\%, 40\%, 60\%, 80\%$, and 100% with $\mathcal{U}_1$, respectively. We measure the distance between the model update of $\mathcal{U}_1$ and those of $\mathcal{U}_2$–$\mathcal{U}_7$ using the proposed sketch-based metric as well as three conventional distance metrics: Euclidean distance (L2), Manhattan distance (L1), and cosine

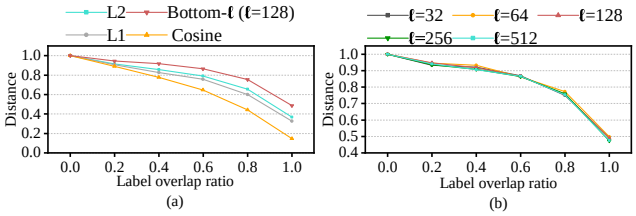


Fig. 7. Effectiveness of the sketch-based similarity metric on MNIST. (a) Comparison with conventional distance metrics under different label-overlap ratios. (b) Similarity estimation with different Bottom-ℓ sketch sizes.

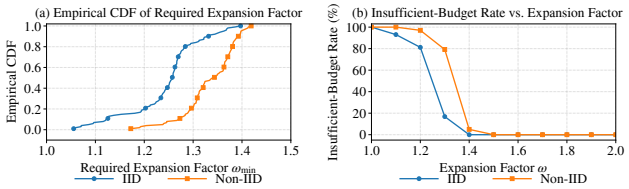


Fig. 8. Sufficiency of the interpolation budget on Fashion-MNIST under IID and Non-IID settings. (a) Empirical CDF of the minimum required expansion factor $\omega_{\min}$. (b) Insufficient-budget rate under different expansion factors ω.

distance. For fair comparison, each metric is normalized by its maximum observed value.

As shown in Fig. 7(a), all evaluated metrics decrease as the label-overlap ratio increases. This shows that clients with more similar data distributions generate more similar model updates. The proposed sketch-based metric follows the same trend as the conventional distance metrics, showing that Bottom-ℓ sketches preserve the similarity relationship among sparsified model updates. Furthermore, Fig. 7(b) shows that different Bottom-ℓ sketch sizes produce nearly identical similarity curves, showing that the similarity estimation is stable under different sketch sizes.

5) Interpolation Budget Sufficiency: To evaluate the sufficiency of the sketch-based interpolation budget, we compare the estimated bucket-level support coverage with the actual coordinate-level union size over 100 communication rounds. Let S^t_{union} denote the number of distinct retained coordinates across all clients in round t. Since the sparsity S^t of the aggregated polynomial satisfies $S^t \le S^t_{\text{union}}$, the latter provides a conservative upper bound on the number of nonzero terms required for sparse recovery. Accordingly, the minimum expansion factor required in round t is $\omega^t_{\min} = S^t_{\text{union}}/(\widehat{S}^t + \delta^t)$.

Fig. 8 reports the results on Fashion-MNIST, with additional results on MNIST and CIFAR-10 provided in Appendix C-D. Fig. 8(a) shows the empirical CDF of $\omega^t_{\min}$ under IID and Non-IID settings, where the CDF at a given ω represents the fraction of rounds satisfying $\omega^t_{\min} \le \omega$. The maximum required expansion factors are approximately 1.40 and 1.42, respectively, both below the adopted setting $\omega = 2$. Fig. 8(b) shows the fraction of rounds in which $\omega(\widehat{S}^t + \delta^t) < S^t_{\text{union}}$. The insufficient-budget rate decreases rapidly with ω, reaches zero for both settings at $\omega = 1.5$, and remains zero thereafter. Thus, with $\omega = 2$, the evaluation budget $2M = 2\omega(\widehat{S}^t + \delta^t)$ provides sufficient evaluations for sparse polynomial recovery in all evaluated rounds. We further set $\rho = 2$ as a conservative expansion factor for repeated recovery if the initial recovery fails verification.

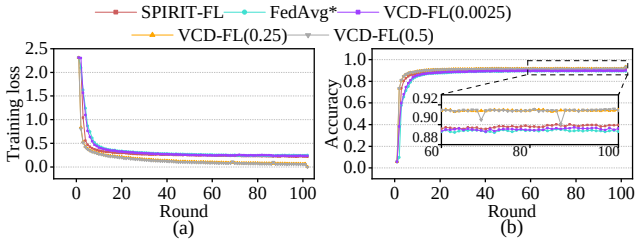


Fig. 9. Model utility comparison among SPIRIT-FL, FedAvg*, and VCD-FL. FedAvg* uses the same Top-k sparsification ratio ($k = 0.0025$) and quantization precision ($\Psi = 6$) as SPIRIT-FL, while VCD-FL is evaluated with sparsification ratios of $k = 0.0025$, 0.25, and 0.5. (a) Training loss. (b) Test accuracy.

D. Model Utility

We evaluate the model utility of SPIRIT-FL on Non-IID Fashion-MNIST using a CNN model. FedAvg* denotes a variant of FedAvg configured with the same Top-k sparsification ratio and quantization precision as SPIRIT-FL, and serves as an ablation baseline to evaluate the effect of the proposed similarity-based weighting mechanism. VCD-FL serves as a representative privacy-preserving and verifiable FL scheme for comparison. Additional results on MNIST, Fashion-MNIST, and CIFAR-10 under both IID and Non-IID settings are provided in Appendix C-F, where similar trends are observed.

As shown in Fig. 9, SPIRIT-FL exhibits stable convergence during training. Compared with FedAvg*, SPIRIT-FL achieves lower training loss while maintaining comparable test accuracy. Since FedAvg* uses the same Top-k sparsification ratio and quantization precision as SPIRIT-FL, these results show that the similarity-based weighting mechanism does not compromise model utility under the same sparsification setting. We further compare SPIRIT-FL with VCD-FL using sparsification ratios of 0.0025, 0.25, and 0.5 for VCD-FL. At the same sparsification ratio ($k = 0.0025$), SPIRIT-FL achieves model utility comparable to VCD-FL. Moreover, despite using a substantially smaller sparsification ratio, SPIRIT-FL with $k = 0.0025$ achieves test accuracy comparable to VCD-FL with $k = 0.25$ and $k = 0.5$. These results demonstrate that SPIRIT-FL preserves competitive model utility while retaining a substantially smaller fraction of model-update entries.

E. Blockchain Scalability

We further evaluate the blockchain scalability of SPIRIT-FL by varying the number of participating clients from 10 to 200. The detailed results, including gas consumption, calldata size, latency, and effective transaction throughput, are reported in Appendix C-E.

VII. Related Work

A. Aggregation for Sparsified Model Updates

Gradient sparsification reduces the communication overhead of federated learning by transmitting only a subset of local model updates [4], [5]. Common approaches include random sparsification and Top-k sparsification. While random sparsification selects coordinates randomly, Top-k sparsification retains the entries with the largest magnitudes. When performed independently by clients, however, Top-k sparsification results in different retained coordinate sets across clients, making sparse aggregation more challenging than aggregation over dense updates with predefined coordinates.

A straightforward approach is to transmit the retained coordinates together with their corresponding model update values, allowing the aggregation server to align sparsified model updates before aggregation [13]. Although this approach preserves the flexibility of independent Top-k sparsification, the revealed coordinates may leak information about clients' local training data [5], [6], [43]. Therefore, existing privacy-preserving approaches seek to protect sparse coordinates during aggregation.

To address this privacy concern, existing approaches mainly follow two directions. One direction applies a shared sparsification mask across clients, ensuring that all sparsified model updates retain the same coordinate set before secure aggregation. Kerkouche *et al.* [44] construct a common Top-k mask using an auxiliary public dataset. Similarly, FedSMP [45] requires clients to share a common sparsification mask, generated either from auxiliary public data for Top-k sparsification or through random coordinate selection. Lu *et al.* [46] construct a globally shared coordinate set from the union of clients' local Top-k sets, while Ergun *et al.* [47] establish pairwise shared random coordinate sets. These approaches make sparse updates compatible with secure aggregation by constraining the coordinate sets used by clients, but limit the flexibility and potential utility of independently selected Top-k sparsification.

Another direction preserves independent sparsification while protecting sparse coordinates using trusted hardware or multi-server architectures. Olive [16] supports native Top-k sparsification using Intel SGX, but relies on trusted hardware. More recently, Clover [5] employs a three-server secret-sharing framework to aggregate independently sparsified model updates while hiding both sparse coordinates and update values, but requires multiple non-colluding servers. In contrast, SPIRIT-FL enables privacy-preserving aggregation of independently sparsified model updates without enforcing a common sparsification pattern or relying on trusted hardware or multiple non-colluding servers.

B. Verifiable Aggregation and Adaptive Weighting

Verifiable aggregation enables clients to verify the correctness of aggregated model updates. Existing schemes employ different techniques to achieve this goal. VCD-FL [39] protects local model updates through pseudorandom masking and verifies aggregation correctness using polynomial commitments, while SWAS [40] employs pseudorandom masking together with polynomial-interpolation-based verification. VLMR-SFL [41] employs homomorphic hashes to authenticate aggregated model updates. MRNIVA [48] realizes non-interactive verification through lightweight multi-client verifiable computation. OPSA [49] relies on trusted execution environments for secure aggregation and verification. VeSAFL [50] distributes aggregation and verification among multiple servers, and DSVFL [42] combines threshold Paillier encryption with aggregated signatures for result verification. Despite their different verification mechanisms, these schemes generally assume that client model updates share a predefined coordinate space and therefore do not consider the correctness of sparse coordinate alignment introduced by independent sparsification.

Similarity-based adaptive aggregation mitigates statistical heterogeneity by assigning different aggregation weights according to relationships among client model updates [22], [23]. FedAA [51] adaptively optimizes client contributions through reinforcement learning, while FedAWA [52] adapts aggregation weights based on client model update directions. CoRe-Fed [53] incorporates embedding alignment into dynamic client weighting. However, these methods typically rely on the aggregation server to evaluate client relationships and determine aggregation weights, assuming that the server faithfully performs the weighting procedure.

In contrast, SPIRIT-FL enables verifiable aggregation over independently sparsified model updates while deriving similarity-based aggregation weights from sparse coordinate structures without relying on the aggregation server for similarity estimation or weight determination.

VIII. Conclusion

This paper presents SPIRIT-FL, to the best of our knowledge, the first sparse-polynomial framework for privacy-preserving and verifiable aggregation in independently sparsified federated learning. Rather than treating privacy-preserving aggregation, verifiable aggregation, and adaptive weighting as separate problems, SPIRIT-FL unifies them through a sparse-polynomial representation. This unified design preserves the flexibility of independent sparsification while enabling secure aggregation, efficient verification, and trustworthy adaptive weighting within a single framework. Security analysis and experimental evaluation demonstrate that SPIRIT-FL enables secure and practical independently sparsified federated learning. An important direction for future work is to extend SPIRIT-FL to Byzantine-robust federated learning. Supporting verifiable aggregation in the presence of malicious clients while preserving the efficiency benefits of independent sparsification remains an open and challenging problem.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282.

[2] J. C. Zhao, S. Bagchi, S. Avestimehr, K. S. Chan, S. Chaterji, D. Dimitriadis, J. Li, N. Li, A. Nourian, and H. R. Roth, "The federation strikes back: A survey of federated learning privacy attacks, defenses, applications, and policy landscape," *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–37, 2025.

[3] H. Asi, V. Feldman, H. Keller, G. N. Rothblum, and K. Talwar, "PREAMBLE: Private and efficient aggregation via block sparse vectors," in *Advances in Neural Information Processing Systems 38*, vol. 38, 2025.

[4] L. Condat, A. Maranjyan, and P. Richtárik, "LoCoDL: Communication-efficient distributed learning with local training and compression," in *Proceedings of the 13th International Conference on Learning Representations*, 2025.

[5] S. Xu, Y. Zheng, and Z. Hua, "Harnessing sparsification in federated learning: A secure, efficient, and differentially private realization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2025, pp. 2354–2368.

[6] J. Du, J. Hu, Z. Wang, P. Sun, N. Z. Gong, K. Ren, and C. Chen, "SoK: On gradient leakage in federated learning," in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3045–3064.

[7] Y. Liu, Z. Jia, Z. Jin, X. Chen, S. Bian, R. Xu, D. Li, J. Liu, and Y. Lu, "Aion: Robust and efficient multi-round single-mask secure aggregation against malicious participants," in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3025–3044.

[8] Y. Ma, J. Woods, S. Angel, A. Polychroniadou, and T. Rabin, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. IEEE, 2023, pp. 477–496.

[9] M. Palazzo, F. W. Dekker, A. Brighente, M. Conti, and Z. Erkin, "Privacy-preserving data aggregation with public verifiability against internal adversaries," in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 6957–6974.

[10] G. N. Rothblum, E. Omri, J. Chen, and K. Talwar, "PINE: Efficient verification of a euclidean norm bound of a Secret-Shared vector," in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 6975–6992.

[11] K. Zhang, S. Cheng, G. Shen, B. Ribeiro, S. An, P.-Y. Chen, X. Zhang, and N. Li, "CENSOR: Defense against gradient inversion via orthogonal subspace bayesian sampling," in *Proceedings of the Network and Distributed System Security Symposium*, 2025.

[12] Y. Ding, P. Mai, J. Zhang, S. S. M. Chow, M. Du, and Y. Pang, "Shared spotlight meridian: Distributed sparse pseudorandom functions for scalable federated learning," in *Proceedings of the 2026 IEEE Symposium on Security and Privacy*. IEEE, 2026, pp. 2269–2287.

[13] H. U. Sami and B. Guler, "Secure gradient aggregation with sparsification for resource-limited federated learning," *IEEE Transactions on Communications*, vol. 72, no. 11, pp. 6883–6899, 2024.

[14] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2017, pp. 1175–1191.

[15] H. Mozaffari, V. Shejwalkar, and A. Houmansadr, "Every vote counts: Ranking-based training of federated learning to resist poisoning attacks," in *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 1721–1738.

[16] F. Kato, Y. Cao, and M. Yoshikawa, "Olive: Oblivious federated learning on trusted execution environment against the risk of sparsification," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2404–2417, 2023.

[17] J. Ma, H. Liu, M. Zhang, and Z. Liu, "VPFL: Enabling verifiability and privacy in federated learning with zero-knowledge proofs," *Knowledge-Based Systems*, vol. 299, p. 112115, 2024.

[18] C. Lee, J. Heiss, S. Tai, and J. W.-K. Hong, "End-to-end verifiable decentralized federated learning," in *Proceedings of the 2024 IEEE International Conference on Blockchain and Cryptocurrency*. IEEE, 2024, pp. 434–442.

[19] Y. Ma, Y. Guo, H. Karthikeyan, and A. Polychroniadou, "Armadillo: Robust single-server secure aggregation for federated learning with input validation," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2025, pp. 2219–2233.

[20] Z. Lu and S. Lu, "LZKSA: Lattice-based special zero-knowledge proofs for secure aggregation's input verification," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2025, pp. 573–587.

[21] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proceedings of the Network and Distributed System Security Symposium*, 2021.

[22] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19 767–19 788.

[23] M. S. Islam, S. Javaherian, F. Xu, X. Yuan, L. Chen, and N.-F. Tzeng, "FedClust: Tackling data heterogeneity in federated learning through weight-driven client clustering," in *Proceedings of the 53rd International Conference on Parallel Processing*. ACM, 2024, pp. 474–483.

[24] C. Zhang, Y. Xu, J. Tan, J. An, and W. Jin, "MingledPie: A cluster mingling approach for mitigating preference profiling in CFL," in *Proceedings of the Network and Distributed System Security Symposium*, 2025.

[25] E. Cohen and H. Kaplan, "Summarizing data using bottom-k sketches," in *Proceedings of the 26th Annual ACM Symposium on Principles of Distributed Computing*. ACM, 2007, pp. 225–234.

[26] C. Jin and Y. Xu, "Shaving logs via large sieve inequality: Faster algorithms for sparse convolution and more," in *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. ACM, 2024, pp. 1573–1584.

[27] M. Ben-Or and P. Tiwari, "A deterministic algorithm for sparse multivariate polynomial interpolation," in *Proceedings of the 20th Annual ACM Symposium on Theory of Computing*. ACM, 1988, pp. 301–309.

[28] L. Pu, J. Gu, C. Lin, and X. Huang, "Janus: Dual-server multi-round secure aggregation with verifiability for federated learning," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 49 927–49 945.

[29] E. Ebrahimi, M. Sober, A.-T. Hoang, C. U. Ileri, W. Sanders, and S. Schulte, "Blockchain-based federated learning utilizing zero-knowledge proofs for verifiable training and aggregation," in *Proceedings of the 2024 IEEE International Conference on Blockchain*. IEEE, 2024, pp. 54–63.

[30] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems 32*, vol. 32, 2019, pp. 14 774–14 784.

[31] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—how easy is it to break privacy in federated learning?" in *Advances in Neural Information Processing Systems 33*, vol. 33, 2020, pp. 16 937–16 947.

[32] K. Yue, R. Jin, C.-W. Wong, D. Baron, and H. Dai, "Gradient obfuscation gives a false sense of security in federated learning," in *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6381–6398.

[33] Y. LeCun, C. Cortes, and C. J. C. Burges, "MNIST handwritten digit database," 2010.

[34] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[35] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[36] X. Wang, D. Dimitriadis, S. Koyejo, and S. Tople, "Invariant aggregator for defending against federated backdoor attacks," in *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 238. PMLR, 2024, pp. 1716–1724.

[37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[38] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, vol. 32, 2019, pp. 8024–8035.

[39] S. Gao, J. Luo, J. Zhu, X. Dong, and W. Shi, "VCD-FL: Verifiable, collusion-resistant, and dynamic federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 3760–3773, 2023.

[40] Y. He and J. Yu, "Toward secure weighted aggregation for privacy-preserving federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 3475–3488, 2025.
[41] J. Li, S. Xu, X. Han, J. Ning, X. He, and G. Xu, "Verifiable and lightweight multi-round secure federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 403–416, 2026.
[42] M. Li, Y. Tian, J. Zhang, J. Xiong, X. Cui, and J. Ma, "Enhancing federated learning in edge computing with secure aggregation and dynamic verification," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 4618–4631, 2026.
[43] L. Chen, D. Xiao, X. Xiao, and Y. Zhang, "Secure and efficient federated learning via novel authenticable multi-party computation and compressed sensing," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 10 141–10 156, 2024.
[44] R. Kerkouche, G. Ács, C. Castelluccia, and P. Genevès, "Constrained differentially private federated learning for low-bandwidth devices," in *Proceedings of the 37th Conference on Uncertainty in Artificial Intelligence*, ser. Proceedings of Machine Learning Research, vol. 161. PMLR, 2021, pp. 1756–1765.
[45] R. Hu, Y. Guo, and Y. Gong, "Federated learning with sparsified model perturbation: Improving accuracy under client-level differential privacy," *IEEE Transactions on Mobile Computing*, vol. 23, no. 8, pp. 8242–8255, 2024.
[46] S. Lu, R. Li, W. Liu, C. Guan, and X. Yang, "Top-k sparsification with secure aggregation for privacy-preserving federated learning," *Computers & Security*, vol. 124, p. 102993, 2023.
[47] I. Ergün, H. U. Sami, and B. Guler, "Sparsified secure aggregation for privacy-preserving federated learning," *arXiv preprint arXiv:2112.12872*, 2021.
[48] Y. Zhu, J. Gong, K. Zhang, and H. Qian, "Malicious-resistant non-interactive verifiable aggregation for federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5600–5616, 2024.
[49] Z. Guan, Y. Zhao, Z. Wan, and J. Han, "OPSA: Efficient and verifiable one-pass secure aggregation with TEE for federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 5, pp. 5321–5334, 2025.
[50] J. Bouamama, Y. Benkaouz, and M. Ouzzif, "VeSAFL: Verifiable secure aggregation for privacy-preserving federated learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6592–6609, 2025.
[51] J. He, W. Chen, and X. Zhang, "FedAA: A reinforcement learning perspective on adaptive aggregation for fair and robust federated learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 16, 2025, pp. 17 085–17 093.
[52] C. Shi, H. Zhao, B. Zhang, M. Zhou, D. Guo, and Y. Chang, "FedAWA: Adaptive optimization of aggregation weights in federated learning using client vectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 30 651–30 660.
[53] N. Mukhtiar, A. Mahmood, and Q. Z. Sheng, "CoRe-Fed: Bridging collaborative and representation fairness via federated embedding distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 29, 2026, pp. 24 458–24 466.
[54] J. Zhang, Y. Liu, Y. Hua, H. Wang, T. Song, Z. Xue, R. Ma, and J. Cao, "PFLlib: A beginner-friendly and comprehensive personalized federated learning library and benchmark," *Journal of Machine Learning Research*, vol. 26, no. 50, pp. 1–10, 2025.
[55] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "SoK: Gradient inversion attacks in federated learning," in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025, pp. 6439–6459.

APPENDIX A
DETAILS OF THE ROOT RECOVERY ALGORITHM

Fig. 10 presents the Cantor–Zassenhaus factorization procedure used to recover the root set of the locator polynomial $\Lambda(z)$. The procedure maintains a queue of polynomials that remain to be factored. Initially, the queue contains only the locator polynomial $\Lambda(z)$, while the root set $\mathcal{Z}$ is initialized as empty. Each iteration removes one polynomial $G(z)$ from the queue. If $G(z)$ is linear, its root is directly extracted and inserted into $\mathcal{Z}$. Otherwise, the Cantor–Zassenhaus splitting procedure is invoked to obtain two nontrivial factors. Specifically, a random polynomial $a(z) \in \mathbb{F}_q[z]$ satisfying $\deg(a) < \deg(G)$ is sampled, and $b(z) = a(z)^{(q-1)/2} \bmod G(z)$ is computed. The algorithm then computes $G_1(z) = \gcd(b(z) - 1, G(z))$. If $1 \le \deg(G_1) < \deg(G)$, a nontrivial factorization is obtained. The remaining factor $G_2(z) = G(z)/G_1(z)$ is computed, and both $G_1(z)$ and $G_2(z)$ are inserted into the queue for further processing. Otherwise, another random polynomial $a(z)$ is sampled, and the splitting procedure is repeated until a nontrivial factorization is obtained.

Input: Locator polynomial $\Lambda(z) \in \mathbb{F}_q[z]$
Output: Root set $\mathcal{Z} = \{z_1, z_2, \ldots, z_S\}$
Initialize $\mathcal{Z} \leftarrow \emptyset$ and $\mathcal{Q} \leftarrow \{\Lambda(z)\}$
while $\mathcal{Q} \neq \emptyset$ **do**
  Pop one polynomial $G(z)$ from $\mathcal{Q}$
  if $\deg(G) = 1$ **then**
    Extract the root of $G(z)$ and add it to $\mathcal{Z}$
  else
    repeat
      Sample $a(z) \in \mathbb{F}_q[z]$ with $\deg(a) < \deg(G)$
      Compute $b(z) = a(z)^{(q-1)/2} \bmod G(z)$
      Compute $G_1(z) = \gcd(b(z) - 1, G(z))$
    until $1 \le \deg(G_1) < \deg(G)$
    Compute $G_2(z) = G(z)/G_1(z)$
    Insert $G_1(z)$ and $G_2(z)$ into $\mathcal{Q}$
  end if
end while
return $\mathcal{Z}$

Fig. 10. Cantor–Zassenhaus factorization for recovering the root set of the locator polynomial.

The procedure terminates when every polynomial remaining in the queue has been reduced to a linear factor. Since the locator polynomial in SPIRIT-FL satisfies $\Lambda(z) = \prod_{r=1}^{S}(z - z_r)$, where all roots are distinct, repeated application of the Cantor–Zassenhaus splitting procedure eventually decomposes $\Lambda(z)$ into all of its linear factors. Therefore, the extracted roots exactly constitute the complete root set $\mathcal{Z} = \{z_1, z_2, \ldots, z_S\}$, which is subsequently used by the AS to recover the corresponding coordinates through the lookup table $\mathcal{T}$ during sparse polynomial recovery.

APPENDIX B
DETAILED PROOF

A. Detailed Privacy Proofs

1) The proof of Theorem 1:

Proof. For an honest client $\mathcal{U}_i$, consider the local evaluation set $\mathcal{E}_i^t$ and the masked evaluation set $\widetilde{\mathcal{E}}_i^t$ generated in the Evaluation Point Uploading step of Section IV-F. For each evaluation point α^m, the uploaded masked evaluation is computed as

$$\widetilde{f}_i(\alpha^m) = f_i(\alpha^m) + \xi_i^m, \tag{25}$$

where ξ_i^m is the pairwise mask generated from the shared seeds between $\mathcal{U}_i$ and other clients:

$$\xi_i^m = \sum_{\mathcal{U}_j \in \mathcal{U},\, i<j} \mathrm{PRG}(s_{ij} \| m \| t) - \sum_{\mathcal{U}_j \in \mathcal{U},\, i>j} \mathrm{PRG}(s_{ij} \| m \| t). \tag{26}$$

Assume that the AS colludes with at most $N-2$ clients. For any honest client $\mathcal{U}_i$, there exists at least one other honest client $\mathcal{U}_{j^*}$ with $j^* \neq i$. Therefore, the pairwise seed s_{ij^*} is unknown to the adversary. Let $\mathbf{r}_{ij^*} = (\mathrm{PRG}(s_{ij^*} \| m \| t))_{m=0}^{2M-1}$ denote the mask sequence generated from this unknown seed. We consider a hybrid experiment in which $\mathbf{r}_{ij^*}$ is replaced by a uniformly random sequence $\mathbf{u} \leftarrow \mathbb{F}_q^{2M}$. By the security of the PRG, the adversarial views in the real and hybrid experiments are computationally indistinguishable.

In the hybrid experiment, after fixing all other mask components known to the adversary, the masked polynomial evaluations of the two honest clients can be expressed as

$$\widetilde{\mathcal{E}}_i^t = \mathcal{E}_i^t + \mathbf{u} + \mathbf{a}_i, \quad \widetilde{\mathcal{E}}_{j^*}^t = \mathcal{E}_{j^*}^t - \mathbf{u} + \mathbf{a}_{j^*}, \tag{27}$$

where $\mathbf{a}_i$ and $\mathbf{a}_{j^*}$ denote the remaining mask components.

Since $\mathbf{u}$ is uniformly random over $\mathbb{F}_q^{2M}$, the distribution of $\widetilde{\mathcal{E}}_i^t$ is independent of the individual polynomial evaluations $\mathcal{E}_i^t$ from the adversary's perspective. However, the random masks cancel when the two honest clients' uploads are aggregated, yielding $\widetilde{\mathcal{E}}_i^t + \widetilde{\mathcal{E}}_{j^*}^t = \mathcal{E}_i^t + \mathcal{E}_{j^*}^t + \mathbf{a}_i + \mathbf{a}_{j^*}$.

Therefore, the adversary can obtain only the aggregate polynomial evaluations of the remaining honest clients, but cannot distinguish how this aggregate is distributed among individual honest clients. More precisely, consider any alternative pair of polynomial evaluations $(\mathcal{E}_i^{t\prime}, \mathcal{E}_{j^*}^{t\prime})$ satisfying $\mathcal{E}_i^{t\prime} + \mathcal{E}_{j^*}^{t\prime} = \mathcal{E}_i^t + \mathcal{E}_{j^*}^t$. A corresponding translation of the random sequence $\mathbf{u}$ produces exactly the same distribution of the masked polynomial evaluations observed by the adversary. Hence, the adversarial view depends on the honest clients' polynomial evaluations only through their aggregate.

Therefore, in the hybrid experiment, the masked polynomial evaluations reveal no additional information about any individual honest client's local polynomial evaluations beyond the aggregate evaluations of the remaining honest clients.

Finally, by the computational indistinguishability between the real and hybrid experiments established by PRG security, the same privacy guarantee holds in the real protocol. □

2) The proof of Theorem 2:

Proof. For each retained coordinate $j \in \pi_i^t$, SPIRIT-FL first maps it into the bucket domain $b_j = j \bmod \beta$, where $b_j \in \mathcal{I}_\beta$. In each communication round t, the public hash function $h^t : \mathcal{I}_\beta \to (0,1)$ is applied to obtain the bucket-level hash value $\nu_j^t = h^t(b_j)$.

Since the hash functions are public, the adversary can observe the published bucket-level hash values contained in the Bottom-ℓ sketch. Therefore, SPIRIT-FL does not attempt to hide bucket-level information. Instead, the privacy goal is to prevent the recovery of the exact retained coordinate within each bucket.

For each bucket $b \in \mathcal{I}_\beta$, let $C_b = \{j \in \{0,\dots,d-1\} : j \bmod \beta = b\}$ denote the set of original coordinates mapped to b. By the modulo mapping, its size satisfies

$$|C_b| \in \left\{ \left\lfloor \frac{d}{\beta} \right\rfloor, \left\lceil \frac{d}{\beta} \right\rceil \right\}. \tag{28}$$

Hence, observing the bucket-level information of a retained coordinate reveals only that the coordinate belongs to C_b, but not which coordinate in C_b is actually retained, since all coordinates in C_b share the same bucket index and produce the same bucket-level hash value under h^t. Therefore, each revealed bucket identifies only a candidate set of approximately d/β original coordinates rather than an exact retained coordinate. This privacy property follows from the many-to-one coordinate-to-bucket mapping, rather than from any assumption on the selection probability of coordinates within a bucket.

Therefore, although the Bottom-ℓ sketch reveals sampled bucket-level hash information and approximate coordinate overlap among clients, it does not uniquely reveal the original Top-k coordinate set. □

3) The proof of Theorem 3:

Proof. For an honest client $\mathcal{U}_i$, its weighted sparsified model update is encoded as $f_i(x) = \sum_{j \in \pi_i^t} \widetilde{\Delta}_i^t[j] x^j$. Let $\mathcal{S}_i = \mathrm{supp}(f_i) = \left\{ j \in \pi_i^t : \widetilde{\Delta}_i^t[j] \neq 0 \right\}$ denote the support of $f_i(x)$, and let $s_i = |\mathcal{S}_i|$. Let $\Gamma_i = \{\tau_1, \tau_2, \dots, \tau_m\}, (m = |\Gamma_i|)$ denote the set of verification points at which evaluations of $f_i(x)$ are disclosed. The corresponding verification leakage is $\mathcal{L}_i^{\mathrm{ver}} = \{(\tau_r, f_i(\tau_r))\}_{r=1}^m$.

We prove a stronger statement by additionally giving the adversary the exact support $\mathcal{S}_i$. Let $\mathcal{S}_i = \{j_1, j_2, \dots, j_{s_i}\}$, and define the coefficient vector $\mathbf{c}_i = \left[\widetilde{\Delta}_i^t[j_1], \widetilde{\Delta}_i^t[j_2], \dots, \widetilde{\Delta}_i^t[j_{s_i}]\right]^\mathsf{T}$. By the definition of $\mathcal{S}_i$, every entry of $\mathbf{c}_i$ is nonzero.

The disclosed evaluations satisfy $\mathbf{y}_i = \mathbf{A}_i \mathbf{c}_i$, where $\mathbf{y}_i = [f_i(\tau_1), f_i(\tau_2), \dots, f_i(\tau_m)]^\mathsf{T}$, and

$$\mathbf{A}_i = \begin{bmatrix} \tau_1^{j_1} & \tau_1^{j_2} & \cdots & \tau_1^{j_{s_i}} \\ \tau_2^{j_1} & \tau_2^{j_2} & \cdots & \tau_2^{j_{s_i}} \\ \vdots & \vdots & \ddots & \vdots \\ \tau_m^{j_1} & \tau_m^{j_2} & \cdots & \tau_m^{j_{s_i}} \end{bmatrix} \in \mathbb{F}_q^{m \times s_i}. \tag{29}$$

Since $m < s_i$, and $\mathrm{rank}(\mathbf{A}_i) \leq m < s_i$ Hence, the null space of $\mathbf{A}_i$ is nontrivial, and there exists a nonzero vector $\mathbf{v} = [v_1, v_2, \dots, v_{s_i}]^\mathsf{T} \in \mathbb{F}_q^{s_i}$ such that $\mathbf{A}_i \mathbf{v} = \mathbf{0}$.

Consider an alternative coefficient vector $\mathbf{c}_i' = \mathbf{c}_i + \lambda \mathbf{v}$, where $\lambda \in \mathbb{F}_q^*$. Since $\mathbf{v} \neq \mathbf{0}$ and $\lambda \neq 0$, we have $\mathbf{c}_i' \neq \mathbf{c}_i$.

For each component with $v_r \neq 0$, the corresponding coefficient $\widetilde{\Delta}_i^t[j_r] + \lambda v_r$ becomes zero for at most one value of λ, namely $\lambda = -\widetilde{\Delta}_i^t[j_r]/v_r$. Therefore, at most s_i nonzero values of λ are forbidden. Since $q - 1 > s_i$, there exists a nonzero $\lambda \in \mathbb{F}_q$ such that every entry of $\mathbf{c}_i'$ remains nonzero.

Using $\mathbf{c}'_i$, define $f'_i(x) = \sum_{r=1}^{s_i} c'_{i,r} x^{j_r}$, where $c'_{i,r}$ denotes the r-th entry of $\mathbf{c}'_i$. Then $f'_i(x) \neq f_i(x)$, while

$$\mathrm{supp}(f'_i) = \mathrm{supp}(f_i) = \mathcal{S}_i.$$

Moreover, $\mathbf{A}_i \mathbf{c}'_i = \mathbf{A}_i(\mathbf{c}_i + \lambda \mathbf{v}) = \mathbf{A}_i \mathbf{c}_i + \lambda \mathbf{A}_i \mathbf{v} = \mathbf{y}_i$. Therefore, $f'_i(\tau_r) = f_i(\tau_r)$, where $r = 1, \ldots, m$. Hence, at least two distinct sparse polynomials with the same exact support $\mathcal{S}_i$ are consistent with all disclosed verification evaluations. Consequently, even if the adversary is additionally given the exact support $\mathcal{S}_i$, the disclosed verification evaluations do not uniquely determine $f_i(x)$. Thus, when $|\Gamma_i| < s_i$ and $q - 1 > s_i$, the disclosed verification evaluations alone are insufficient to uniquely reconstruct the weighted sparsified model update. □

4) The proof of Theorem 4:

Proof. Let $\mathcal{C}$ denote the set of clients colluding with the AS, and let $\mathcal{H} = \mathcal{U} \setminus \mathcal{C}$ denote the set of non-colluding clients. The colluding clients are assumed to be protocol-following adversaries that honestly execute the protocol while revealing their protocol states to the AS. Since the AS colludes with at most $N - 2$ clients, we have $|\mathcal{H}| \geq 2$.

For communication round t, we define the explicitly characterized leakage as

$$\mathcal{L}^t = \left(\mathsf{state}^t_{\mathcal{C}}, F^t_{\mathcal{H}}(x), \mathcal{L}^t_{\mathrm{sk}}, \mathcal{L}^t_{\mathrm{ver}}, pp\right), \tag{30}$$

where $\mathsf{state}^t_{\mathcal{C}}$ denotes the protocol states revealed by the colluding clients, including their sparse polynomials, pairwise seeds shared with other clients, and other protocol information available to them during execution. $F^t_{\mathcal{H}}(x)$ denotes the aggregated sparse polynomial of the non-colluding clients:

$$\mathcal{F}^t_{\mathcal{H}}(x) = \sum_{\mathcal{U}_i \in \mathcal{H}} f^t_i(x). \tag{31}$$

The aggregated sparse polynomial is included in the leakage because it represents the intended aggregation output rather than the individual contributions of non-colluding clients. $\mathcal{L}^t_{\mathrm{sk}}$ and $\mathcal{L}^t_{\mathrm{ver}}$ denote the information explicitly revealed by the public Bottom-ℓ sketches and verification procedures, respectively, and pp denotes the public protocol parameters.

We construct a simulator Sim that generates a view computationally indistinguishable from the real adversarial view using only $\mathcal{L}^t$. First, the simulator directly reproduces the public parameters, the protocol states of colluding clients, the public Bottom-ℓ sketches, and the disclosed verification information from $\mathcal{L}^t$. Since the local sparse polynomials of colluding clients are contained in $\mathsf{state}^t_{\mathcal{C}}$, the simulator can compute their aggregated sparse polynomial. Therefore, the complete aggregated sparse polynomial can be obtained as $\mathcal{F}^t(x) = \mathcal{F}^t_{\mathcal{H}}(x) + \sum_{\mathcal{U}_i \in \mathcal{C}} f^t_i(x)$. It remains to simulate the masked evaluation uploads generated by the non-colluding clients. Let $\mathcal{H} = \{\mathcal{U}_{h_1}, \ldots, \mathcal{U}_{h_r}\}$, where $r = |\mathcal{H}| \geq 2$. For each aggregation evaluation point α^m, the simulator computes the aggregate unmasked evaluation of the non-colluding clients as $Y^t_{\mathcal{H},m} = \mathcal{F}^t_{\mathcal{H}}(\alpha^m)$.

In the hybrid experiment, the outputs generated by the PRG instances derived from pairwise seeds shared exclusively between non-colluding clients are replaced by uniformly random values over $\mathbb{F}_q$. Due to the security of the PRG, the adversarial views in the real experiment and the hybrid experiment are computationally indistinguishable. For a fixed evaluation point α^m, the mask of each non-colluding client $\mathcal{U}_{h_v}$ can be decomposed as $\xi^m_{h_v} = M^m_{h_v} + A^m_{h_v}$, where $M^m_{h_v}$ denotes the mask component generated from pairwise seeds shared exclusively among non-colluding clients, and $A^m_{h_v}$ denotes the mask component generated from pairwise seeds shared between $\mathcal{U}_{h_v}$ and colluding clients.

Since each pairwise mask term shared between two non-colluding clients appears once with a positive sign and once with a negative sign in their respective mask components, we have $\sum_{v=1}^{r} M^m_{h_v} = 0$.

In the hybrid experiment, the vector $(M^m_{h_1}, \ldots, M^m_{h_r})$ is uniformly distributed over the zero-sum subspace of $\mathbb{F}^r_q$, because the pairwise mask terms are replaced by independent uniform random values. Therefore, the masked evaluations of the non-colluding clients are uniformly distributed subject only to their aggregate value and the mask components known by the colluding parties.

The values of $A^m_{h_v}$ are known to the simulator because the corresponding pairwise seeds are revealed by the colluding clients and are included in $\mathsf{state}^t_{\mathcal{C}}$. $\mathsf{state}^t_{\mathcal{C}}$.

Let $\widetilde{Y}^m_{h_v}$ denote the masked evaluation uploaded by client $\mathcal{U}_{h_v}$ at evaluation point α^m. The simulator samples $\widetilde{Y}^m_{h_1}, \ldots, \widetilde{Y}^m_{h_{r-1}}$ uniformly from $\mathbb{F}_q$ and determines the final masked evaluation as $\widetilde{Y}^m_{h_r} = Y^t_{\mathcal{H},m} + \sum_{v=1}^{r} A^m_{h_v} - \sum_{v=1}^{r-1} \widetilde{Y}^m_{h_v}$.

By construction, the simulated masked evaluations satisfy the same aggregate constraint as the real masked evaluations of the non-colluding clients. Moreover, they are identically distributed in the hybrid experiment because the simulated mask components are sampled from the same zero-sum subspace.

Applying the above simulation procedure to all aggregation evaluation points produces a complete masked-evaluation transcript that is identically distributed to the hybrid transcript.

Finally, according to the security of the PRG, the hybrid transcript is computationally indistinguishable from the real transcript. Therefore, the simulator generates a view satisfying

$$\mathsf{View}^t_{\mathcal{A}} \approx_c \mathsf{Sim}(\mathcal{L}^t). \tag{32}$$

The simulation is conditioned on the explicitly characterized leakage $\mathcal{L}^t_{\mathrm{sk}}$ and $\mathcal{L}^t_{\mathrm{ver}}$. These quantities may contain information correlated with individual sparse polynomials and are therefore explicitly included in the leakage rather than assumed to be independently private. Theorems 2 and 3 separately characterize the information revealed by the public Bottom-ℓ sketches and verification procedures.

Consequently, conditioned on the explicitly characterized leakage, the remaining protocol transcript reveals no additional information about the individual sparse polynomials of the non-colluding clients. □

B. Detailed Correctness proof

The proof of Theorem 5

Proof. We prove the correctness of SPIRIT-FL in three steps.

Step 1: Correctness of masked polynomial aggregation. In the Evaluation Point Uploading phase, for each public evaluation point α^m $(m = 0, \ldots, 2M-1)$, each client $\mathcal{U}_i$ uploads the masked evaluation $\widetilde{f}_i(\alpha^m) = f_i(\alpha^m) + \xi_i^m$, where ξ_i^m denotes the pairwise mask defined in the Evaluation Point Uploading phase. For every pair of clients, the mask derived from their shared seed appears with opposite signs in their uploaded masked evaluations. Therefore, when the AS aggregates all uploaded masked evaluations, all pairwise masks cancel out, yielding

$$\sum_{i=1}^{N} \widetilde{f}_i(\alpha^m) = \sum_{i=1}^{N} f_i(\alpha^m). \tag{33}$$

By the definition of Y_m in Eq. 15,

$$\sum_{i=1}^{N} f_i(\alpha^m) = \mathcal{F}(\alpha^m) = Y_m, \qquad m = 0, \ldots, 2M-1. \tag{34}$$

Hence, the AS obtains the evaluation sequence $\{Y_m\}_{m=0}^{2M-1}$ of the aggregated sparse polynomial $\mathcal{F}(x)$. If the initial recovery fails verification, the clients repeat the Evaluation Point Uploading procedure with $2\rho M$ evaluation points. Since the same pairwise masking mechanism is applied to each re-uploaded evaluation, the pairwise masks cancel in the same manner, and the AS obtains the corresponding evaluation sequence $\{Y_m\}_{m=0}^{2\rho M-1}$ of $\mathcal{F}(x)$.

Step 2: Correctness of aggregated sparse polynomial recovery. Using the evaluation sequence $\{Y_m\}_{m=0}^{2M-1}$ obtained in Step 1, the AS performs aggregated sparse polynomial recovery according to the Aggregated Sparse Polynomial Recovery procedure. The aggregated sparse polynomial has the form $\mathcal{F}(x) = \sum_{r=1}^{S} c_r x^{e_r}$, where S denotes the number of nonzero terms, c_r denotes the corresponding coefficient, and e_r denotes the corresponding coordinate index. Let $z_r = \alpha^{e_r}$. Because the multiplicative order of α is at least the coordinate domain size d, the values z_r corresponding to different coordinate indices are distinct.

If the initial evaluation budget satisfies $2M \geq 2S$, the AS obtains sufficient evaluations to recover the aggregated sparse polynomial in the initial recovery attempt. If the initial recovery fails verification, the repeated Evaluation Point Uploading procedure is triggered. Under the condition $2\rho M \geq 2S$, the re-uploaded evaluation sequence likewise contains sufficient evaluations for sparse polynomial recovery. Hence, in either case, the recovery procedure operates on at least $2S$ evaluations. Moreover, the Hankel matrix in Eq. 16 has rank S because it admits a Vandermonde factorization. Since the values z_r are distinct and every coefficient c_r is nonzero, the corresponding Vandermonde matrix has full column rank. Therefore, the AS correctly determines the sparsity S.

Let $\{Y_m\}_{m=0}^{2L-1}$ denote the evaluation sequence used in the successful recovery attempt, where $L = M$ for the initial recovery and $L = \rho M$ for the repeated recovery. The evaluation sequence $\{Y_m\}_{m=0}^{2L-1}$ satisfies the linear recurrence corresponding to the locator polynomial $\Lambda(z)$ in Eq. 17. By solving the Hankel linear system in Eq. 18, the AS correctly obtains $\Lambda(z)$. Factoring $\Lambda(z)$ over $\mathbb{F}_q$ yields the root set $\{z_1, z_2, \ldots, z_S\}$. Since $z_r = \alpha^{e_r}$ and the lookup table $\mathcal{T}$ is precomputed during the initialization phase, the corresponding coordinate index e_r is recovered by table lookup for each root. Finally, the Vandermonde system in Eq. 19 uniquely recovers the coefficients $c_1, c_2, \ldots, c_S$ because the values z_r are distinct. Since q is sufficiently large to avoid modular wrap-around of the aggregated quantized coefficients, the recovered coefficients uniquely correspond to their intended signed integer representations. Therefore, the AS correctly recovers the aggregated sparse polynomial $\mathcal{F}(x)$.

Step 3: Correctness of aggregation verification. After correctly recovering the aggregated sparse polynomial $\mathcal{F}(x)$, the AS broadcasts it to all clients for aggregation verification.

From Step 2, $\mathcal{F}(x) = \sum_{i=1}^{N} f_i(x)$. Evaluating both sides at the public verification point κ^t gives $\mathcal{F}(\kappa^t) = \sum_{i=1}^{N} f_i(\kappa^t)$. Therefore, Eq. 20 holds and the public-point verification succeeds. For each honest client $\mathcal{U}_i$, the revealed verification point γ_i^t and evaluation $f_i(\gamma_i^t)$ are exactly the values used to generate $\mathsf{Com}_i^t = \mathsf{Hash}(\gamma_i^t \| f_i(\gamma_i^t))$. Hence, the commitment opening check succeeds for every honest client. Moreover, evaluating $\mathcal{F}(x) = \sum_{j=1}^{N} f_j(x)$ at each private verification point γ_i^t gives

$$\mathcal{F}(\gamma_i^t) = \sum_{j=1}^{N} f_j(\gamma_i^t), \qquad i = 1, \ldots, N. \tag{35}$$

Therefore, Eq. 21 holds for every private verification point, and the private-point verification succeeds.

Since the commitment opening checks, public-point verification, and private-point verification all succeed, Rule 1 of the Verification Decision accepts $\mathcal{F}(x)$. Consequently, all honest clients accept the aggregation result. □

C. Detailed Verifiability Proofs

1) The proof of Theorem 6:

Proof. Let $\widehat{\mathcal{F}}(x) = \mathcal{F}(x) + \Phi(x)$, where $\Phi(x) \neq 0$ denotes the deviation polynomial introduced by the incorrect aggregation result. According to the protocol order, the AS must first broadcast $\widehat{\mathcal{F}}(x)$ before the clients disclose the verification evaluations $f_i(\kappa^t)$. Under the weak attack model, the AS does not know the common verification point κ^t when generating $\widehat{\mathcal{F}}(x)$. Therefore, the deviation polynomial $\Phi(x)$ is fixed independently of κ^t.

Before public-point verification, each client checks that all exponents in $\widehat{\mathcal{F}}(x)$ belong to the valid coordinate domain $\mathcal{I}_d = \{0, 1, \ldots, d-1\}$; otherwise, the returned polynomial is rejected directly. Therefore, for any incorrect polynomial that reaches the public-point verification stage, both $\widehat{\mathcal{F}}(x)$ and $\mathcal{F}(x)$ have degree at most $d-1$, and hence $\deg(\Phi) \leq d-1$.

According to Eq. 20, the returned polynomial passes the public-point verification only if $\widehat{\mathcal{F}}(\kappa^t) = \sum_{i=1}^{N} f_i(\kappa^t)$. Since

TABLE III
TRAINING HYPERPARAMETERS USED IN THE EVALUATION

Parameter	Value
Optimizer	SGD
Learning rate	0.1
Communication rounds	100 (MNIST, Fashion-MNIST) 200 (CIFAR-10)
Local epochs	5
Mini-batch size	64
Number of participating clients (N)	20
Top-k sparsification ratio	0.25%
Dirichlet concentration α	1

$\mathcal{F}(\kappa^t) = \sum_{i=1}^{N} f_i(\kappa^t)$, this condition is equivalent to $\widehat{\mathcal{F}}(\kappa^t) = \mathcal{F}(\kappa^t)$, and hence $\Phi(\kappa^t) = 0$. A nonzero polynomial over $\mathbb{F}_q$ has at most as many roots as its degree. Since κ^t is uniformly sampled from $\mathbb{F}_q$ independently of the fixed deviation polynomial $\Phi(x)$,

$$\Pr[\Phi(\kappa^t) = 0] \leq \frac{\deg(\Phi)}{q} \leq \frac{d-1}{q}. \tag{36}$$

Therefore, an incorrect aggregated sparse polynomial passes the public-point verification with probability at most $(d-1)/q$ and is detected with probability at least $1-(d-1)/q$. □

2) The proof of Theorem 7:

Proof. Let $\widehat{\mathcal{F}}(x) = \mathcal{F}(x) + \Phi(x)$, where $\Phi(x) \neq 0$ is the deviation polynomial. Under the strong attack model, the AS may obtain the common verification point κ^t from colluding clients. Therefore, it may deliberately construct the deviation polynomial such that $\Phi(\kappa^t) = 0$, allowing the incorrect aggregated sparse polynomial to pass the public-point verification.

Consider an arbitrary non-colluding client $\mathcal{U}_i$. By the strong attack model, the private verification point γ_i of the non-colluding client is unavailable to the AS when it constructs the incorrect aggregated sparse polynomial. Hence, the deviation polynomial $\Phi(x)$ is fixed independently of γ_i^t.

According to the private-point verification rule in Eq. 21, the returned aggregated sparse polynomial passes the private-point verification only if $\widehat{\mathcal{F}}(\gamma_i^t) = \sum_{j=1}^{N} f_j(\gamma_i^t)$.

Since $\mathcal{F}(\gamma_i^t) = \sum_{j=1}^{N} f_j(\gamma_i^t)$, the above equality is equivalent to $\widehat{\mathcal{F}}(\gamma_i^t) = \mathcal{F}(\gamma_i^t)$. Substituting $\widehat{\mathcal{F}}(x) = \mathcal{F}(x) + \Phi(x)$ into the above equality gives $\Phi(\gamma_i^t) = 0$.

Since the exponents of both $\mathcal{F}(x)$ and $\widehat{\mathcal{F}}(x)$ correspond to coordinates in the coordinate domain $\mathcal{I}_d = \{0, 1, \ldots, d-1\}$, the deviation polynomial satisfies $\deg(\Phi) \leq d-1$

A nonzero polynomial over $\mathbb{F}_q$ has at most as many roots as its degree. Since γ_i^t is uniformly selected from $\mathbb{F}_q$ and is independent of $\Phi(x)$,

$$\Pr[\Phi(\gamma_i^t) = 0] \leq \frac{\deg(\Phi)}{q} \leq \frac{d-1}{q}. \tag{37}$$

Therefore, $\Pr[\Phi(\gamma_i^t) \neq 0] \geq 1 - \frac{d-1}{q}$. Since the aggregation result is accepted only if the private-point verification succeeds at every private verification point, failure at the private verification point of any non-colluding client causes the incorrect aggregated sparse polynomial to be rejected. Hence, even if the incorrect aggregated sparse polynomial passes the public-point verification, it passes the private-point verification with probability at most $(d-1)/q$. Equivalently, the private-point verification detects the incorrect aggregated sparse polynomial with probability at least $1 - \frac{d-1}{q}$, which is overwhelming when $(d-1)/q$ is negligible in the security parameter. □

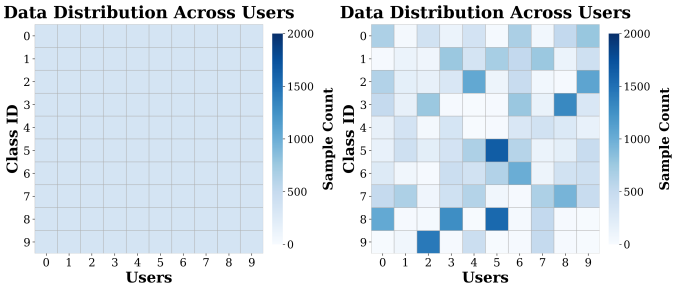


Fig. 11. Representative class distributions across participating clients under IID and Non-IID data partitions. Darker cells indicate that a client holds more training samples from the corresponding class.

TABLE IV
MODEL ARCHITECTURES USED IN THE EVALUATION

Dataset	Model	Layer Configuration
MNIST	CNN	Conv$(1, 32, 5)$–ReLU–MaxPool(2) Conv$(32, 64, 5)$–ReLU–MaxPool(2) FC$(1024, 512)$–ReLU–FC$(512, 10)$
Fashion-MNIST	CNN	Conv$(1, 32, 5)$–ReLU–MaxPool(2) Conv$(32, 64, 5)$–ReLU–MaxPool(2) FC$(1024, 512)$–ReLU–FC$(512, 10)$
CIFAR-10	ResNet-10	Initial Conv–4 Residual Stages (one BasicBlock per stage) Global Average Pooling–FC(10)

APPENDIX C
EXPERIMENTAL DETAILS

A. Datasets and Model Architectures

We evaluate SPIRIT-FL on three standard image-classification benchmarks commonly used in FL studies [54].

- **MNIST** [33] contains 60,000 training images and 10,000 test images of handwritten digits from 10 classes. Each image has a resolution of 28×28 pixels.
- **Fashion-MNIST** [34] contains 60,000 training images and 10,000 test images from 10 fashion categories. Each image has a resolution of 28×28 pixels.
- **CIFAR-10** [35] contains 50,000 training images and 10,000 test images from 10 object classes. Each image has a resolution of 32×32 pixels with three color channels.

For each dataset, we consider both IID and Non-IID data partitions. Under the IID setting, the training samples are randomly and approximately uniformly partitioned across participating clients. Under the Non-IID setting, client data are partitioned using a Dirichlet distribution with concentration parameter $\alpha = 1$. Fig. 11 illustrates representative class distributions across participating clients under IID and Non-IID data partitions.

TABLE V
QUANTITATIVE RECONSTRUCTION QUALITY ON MNIST.

Leakage Setting	MSE ↓	PSNR ↑ (dB)
Full Gradient	0.000388 ± 0.000197	34.82 ± 2.77
Top-k Gradient	0.007086 ± 0.001782	21.63 ± 1.08
Exact Top-k Indices	0.041487 ± 0.022285	15.38 ± 5.03
Bucket-level Support	0.395691 ± 0.022649	4.03 ± 0.25
Bottom-ℓ Sketch	0.103562 ± 0.046047	10.21 ± 1.73

Note: Results are reported as mean $\pm$ standard deviation over 40 randomly selected MNIST samples. Lower MSE and higher PSNR indicate better reconstruction quality.

For MNIST and Fashion-MNIST, we use a CNN consisting of two convolutional layers followed by two fully connected layers. For CIFAR-10, we use ResNet-10 [37], consisting of four residual stages with one BasicBlock in each stage. Table IV summarizes the model architectures.

Unless otherwise specified, all training experiments use the default configuration summarized in Table III. Each training experiment is repeated 10 times with different random seeds, and the reported results are averaged across these runs. The same training configuration is used for all compared schemes whenever the corresponding parameter is applicable.

B. Evaluation of Coordinate Privacy

Gradient information can be exploited to reconstruct clients' local training data [30], [55]. To evaluate the coordinate leakage considered in Theorem 2, we conduct a support-based reconstruction attack on MNIST adapted from the Deep Leakage from Gradients (DLG) framework [30]. MNIST provides a favorable setting for visualizing reconstruction leakage due to its simple and distinctive image structures. We use single-sample gradients to avoid the additional mixing introduced by multiple local updates and focus on the relative leakage caused by exact coordinates, bucket-level support, and Bottom-ℓ sketches.

1) Support-Based Reconstruction Attack: For observations that do not reveal gradient values, we reconstruct a dummy input $\widetilde{\mathbf{x}}$ by matching the exposed support information. Given model parameters $\boldsymbol{\theta}$ and label y, the dummy gradient is

$$\widetilde{g} = \nabla F(f_\theta(\widetilde{\mathbf{x}}), y) \in \mathbb{R}^d. \tag{38}$$

Let $K = kd$ denote the number of coordinates retained by Top-k sparsification. Since Top-k selection is non-differentiable, we approximate the membership of coordinate j using

$$p_j = \sigma\left(\frac{|\widetilde{g}_j|/\mu - \tau_K}{T}\right), \qquad \mu = \frac{1}{d}\sum_{r=1}^{d}|\widetilde{g}_r|, \tag{39}$$

where τ_K is the K-th largest normalized gradient magnitude, T is the temperature parameter, and $\sigma(\cdot)$ denotes the sigmoid function.

For bucket-level observations, coordinate j is mapped to $b(j) = j \bmod \beta$. The differentiable occupancy score of bucket b is defined as $q_b = 1 - \exp\left(-\sum_{j:b(j)=b} p_j\right)$, which represents whether bucket b contains at least one candidate Top-k coordinate without identifying the corresponding coordinate.

We define the support-matching loss as

$$F_{\mathrm{m}}(\mathcal{P}, \mathcal{Q}; \zeta) = -\frac{1}{|\mathcal{P}|}\sum_{u\in\mathcal{P}}\log(\zeta_u+\epsilon) - \frac{1}{|\mathcal{Q}|}\sum_{u\in\mathcal{Q}}\log(1-\zeta_u+\epsilon), \tag{40}$$

where $\mathcal{P}$ and $\mathcal{Q}$ denote the positive and hard-negative sets, respectively, ζ_u is the corresponding membership or occupancy score, and $\epsilon > 0$ is a numerical-stability constant.

For exact Top-k indices, $\mathcal{P} = \mathcal{I}_{\mathrm{top}}$ is the exposed Top-k coordinate set, $\mathcal{Q} = \mathcal{H}$ contains hard-negative coordinates outside $\mathcal{I}_{\mathrm{top}}$, and $\zeta = p$. For bucket-level support, $\mathcal{P} = \mathcal{B}$ is the exposed occupied-bucket set, $\mathcal{Q} = \mathcal{H}_B$ contains hard-negative buckets outside $\mathcal{B}$, and $\zeta = q$.

For the Bottom-ℓ sketch, since $h^t(\cdot)$ is public and the bucket domain $\mathcal{I}_\beta$ is finite, the attacker can enumerate the bucket domain and identify the candidate buckets corresponding to the disclosed Bottom-ℓ hash values. Accordingly, $\mathcal{P} = \mathcal{B}^+$ contains the sampled occupied buckets identified from the sketch. The negative bucket candidates are given by

$$\mathcal{B}^- = \left\{b \in \mathcal{I}_\beta : h^t(b) < \tau_h,\, h^t(b) \notin \mathcal{S}_\ell\right\}, \tag{41}$$

where $\mathcal{S}_\ell$ denotes the disclosed Bottom-ℓ sketch and $\tau_h = \max \mathcal{S}_\ell$ is its hash threshold. The hard-negative set is then selected as $\mathcal{Q} = \mathcal{H}^- \subseteq \mathcal{B}^-$, with $\zeta = q$.

Finally, the dummy input is optimized as

$$\widetilde{\mathbf{x}}^* = \arg\min_{\widetilde{\mathbf{x}}}\left[F_{\mathrm{obs}} + \lambda_c\left(\frac{\sum_{j=1}^{d} p_j - K}{K}\right)^2 + \lambda_{\mathrm{TV}}\,\mathrm{TV}(\widetilde{\mathbf{x}})\right], \tag{42}$$

where $F_{\mathrm{obs}} = F_{\mathrm{m}}(\mathcal{P}, \mathcal{Q}; z)$ is instantiated according to the exposed information, λ_c controls the support-cardinality constraint, and λ_{TV} is the weight of the total-variation regularizer.

2) Results: Fig. 12 and Table V compare reconstruction under five leakage settings. The full and Top-k sparse gradients retain substantial information about the input. When only the exact Top-k indices are exposed, reconstruction quality decreases to an MSE of 0.0415 and a PSNR of 15.38 dB, but recognizable structure remains, showing that sparse coordinates alone can provide useful information for reconstruction.

In comparison, bucket-level support and the Bottom-ℓ sketch yield PSNRs of 4.03 dB and 10.21 dB, respectively, and produce substantially less recognizable reconstructions. Although these two coarse-grained leakage settings do not exhibit a strictly monotonic relationship in reconstruction quality, both provide substantially less useful information for support-based reconstruction than exact Top-k coordinate exposure. These results are consistent with Theorem 2: the Bottom-ℓ sketch does not directly expose the exact Top-k coordinate pattern.

C. Efficiency Evaluation

1) Communication Overhead Analysis: In VCD-FL and SWAS, each parameter group contains $M' - 1$ model parameters and one verification value. Therefore, every parameter group transmits M' values. For a model with dimension d, the communication overhead is

$$C_{\mathrm{VCD/SWAS}} = \frac{32M'\lceil d/(M'-1)\rceil}{8\times 1024^2}\ \mathrm{MB}.$$

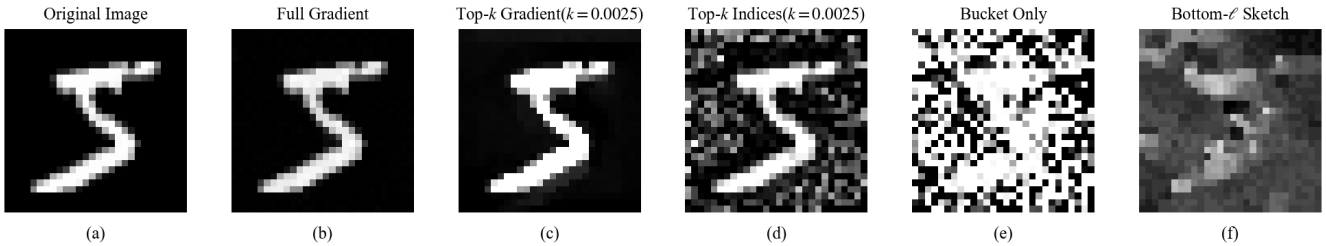


Fig. 12. Reconstruction results under different levels of gradient-coordinate leakage: (a) original input; (b) full gradient; (c) Top-k sparse gradient; (d) exact Top-k indices; (e) bucket-level support; and (f) Bottom-ℓ sketch.

For the same grouping size M', VCD-FL and SWAS have identical communication overhead because both schemes adopt the same parameter-grouping mechanism. Increasing M' reduces the relative verification overhead since more model parameters share one verification value.

In VLMR-SFL, each client transmits a full-dimensional masked model update together with homomorphic-hash-based verification information. Accordingly, the communication overhead is

$$C_{\mathrm{VLMR}} = \frac{32d}{8 \times 1024^2} + \frac{34.4}{1024} \text{ MB}. \tag{43}$$

The first term corresponds to the transmitted masked model update, whereas the second term corresponds to the verification information.

In DSVFL protocol, each client retains $k_d = rd$, gradient entries through Top-k sparsification, where r denotes the Top-k sparsification ratio. Each retained gradient is encrypted using the Paillier cryptosystem before transmission. Under a 1024-bit Paillier security parameter, each encrypted gradient occupies approximately 256 bytes. Therefore, the communication overhead is

$$C_{\mathrm{DSVFL}} = \frac{256k_d}{1024^2} \text{ MB}.$$

For a fair comparison, all schemes employing Top-k sparsification use the same Top-k sparsification ratio as SPIRIT-FL, i.e., $r = 0.0025$.

In SPIRIT-FL, each client transmits masked polynomial evaluations for sparse polynomial recovery, a Bottom-ℓ sketch, a commitment, and the information required for aggregation verification.

During public-point verification, each client $\mathcal{U}_i$ discloses one evaluation $f_i(\kappa^t)$. During private-point verification, $\mathcal{U}_i$ reveals its private verification point γ_i^t and, after the evaluations from the other clients have been fixed, reveals its corresponding evaluation $f_i(\gamma_i^t)$. In addition, for each private verification point γ_j^t revealed by another client $\mathcal{U}_j$ $(j \neq i)$, client $\mathcal{U}_i$ broadcasts the evaluation $f_i(\gamma_j^t)$. Therefore, each client transmits $N + 2$ field elements during the complete verification procedure: one public-point evaluation, $N - 1$ evaluations at the private verification points of the other clients, one private verification point $\gamma_i^t \in \mathbb{F}_q$, and one evaluation at its own private verification point.

Let m denote the number of masked polynomial evaluations transmitted by each client for sparse polynomial recovery in the normal execution of one communication round. Each Bottom-ℓ sketch contains ℓ 64-bit hash values and therefore occupies 8ℓ bytes. The commitment Com_i^t is generated using Keccak-256 and occupies 32 bytes. We report SPIRIT-FL using both 32-bit and 128-bit finite-field representations. Accordingly, the per-client communication overhead is

$$C_{\mathrm{SPIRIT}}^{32} = 4(m + N + 2) + 8\ell + 32 \text{ Bytes}, \tag{44}$$

and

$$C_{\mathrm{SPIRIT}}^{128} = 16(m + N + 2) + 8\ell + 32 \text{ Bytes}. \tag{45}$$

Here, m accounts for the masked polynomial evaluations used for sparse recovery, while the additional $N + 2$ field elements account for the information transmitted during aggregation verification.

In the normal execution of SPIRIT-FL, $m = 2M$. If the initial recovery fails verification, the repeated recovery procedure requires each client to additionally upload $2\rho M$ newly masked polynomial evaluations. Therefore, a round in which repeated recovery is triggered incurs an additional communication cost of

$$C_{\mathrm{repeat}}^{32} = 4(2\rho M) \text{ Bytes}, \tag{46}$$

or

$$C_{\mathrm{repeat}}^{128} = 16(2\rho M) \text{ Bytes}, \tag{47}$$

under the 32-bit and 128-bit field settings, respectively. The communication results reported in Table I correspond to the normal execution without repeated recovery.

In our default experimental setting, $N = 20$ and $\ell = 128$. Thus, each client transmits 22 field elements during aggregation verification, and the Bottom-ℓ sketch occupies $8\ell = 1024$ bytes. The normal per-client communication overheads therefore become

$$C_{\mathrm{SPIRIT}}^{32} = 4(m + 22) + 1056 \text{ Bytes}, \tag{48}$$

and

$$C_{\mathrm{SPIRIT}}^{128} = 16(m + 22) + 1056 \text{ Bytes}. \tag{49}$$

Substituting the model dimensions and the hyperparameters listed in Table III into the above equations yields the communication overhead reported in Table I. Specifically, SPIRIT-FL uploads 4,634 and 4,418 masked polynomial evaluations

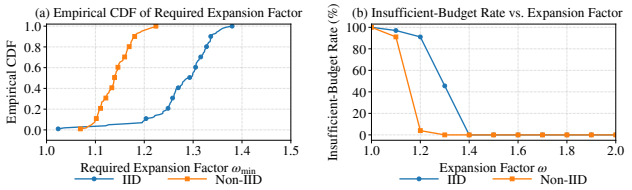


Fig. 13. Interpolation-budget evaluation on MNIST under IID and Non-IID settings. (a) Empirical CDF of the required expansion factor $\omega_{\min}$. (b) Insufficient-budget rate under different expansion factors ω.

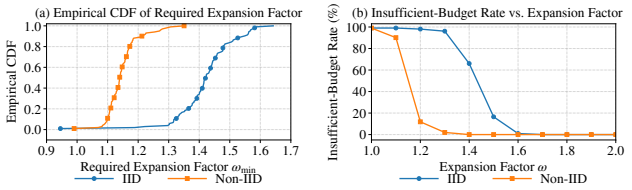


Fig. 14. Interpolation-budget evaluation on CIFAR-10 under IID and Non-IID settings. (a) Empirical CDF of the required expansion factor $\omega_{\min}$. (b) Insufficient-budget rate under different expansion factors ω.

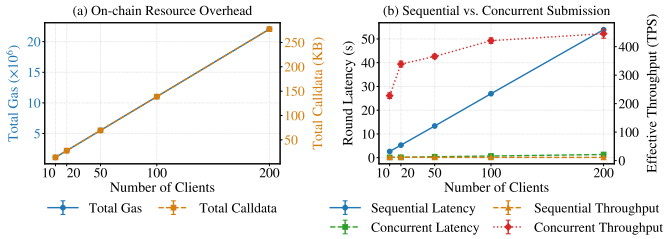


Fig. 15. Blockchain scalability of SPIRIT-FL with different numbers of participating clients under sequential and phase-wise concurrent transaction submission. Error bars represent the standard deviation over five runs.

for MNIST and Fashion-MNIST, respectively, and 30,538 masked polynomial evaluations for CIFAR-10 with ResNet-10. Under the 32-bit setting, the corresponding communication overheads are approximately 0.019 MB, 0.018 MB, and 0.118 MB for MNIST, Fashion-MNIST, and CIFAR-10, respectively. Under the 128-bit setting, the corresponding communication overheads are approximately 0.072 MB, 0.069 MB, and 0.468 MB, respectively.

D. Additional Results on Interpolation Budget Sufficiency

1) MNIST: We further evaluate the sufficiency of the sketch-based interpolation budget on MNIST under both IID and Non-IID settings. For MNIST with the CNN model, we set the bucket parameter to $\beta = 10{,}000$, consistent with the setting used for Fashion-MNIST.

Fig. 13(a) shows the empirical CDF of the required expansion factor $\omega^t_{\min}$. The maximum required expansion factors are approximately 1.38 and 1.22 under the IID and Non-IID settings, respectively, both below the adopted setting $\omega = 2$. Fig. 13(b) further shows that the insufficient-budget rate decreases rapidly as ω increases, reaching zero at $\omega = 1.4$ for IID and $\omega = 1.3$ for Non-IID. Thus, with the adopted $\omega = 2$, no insufficient-budget case is observed under either setting. These results are consistent with those on Fashion-MNIST and further confirm the sufficiency of the adopted interpolation budget.

2) CIFAR-10: We further evaluate the interpolation-budget sufficiency on CIFAR-10 under both IID and Non-IID settings using the same evaluation procedure. For CIFAR-10 with ResNet-10, we set the bucket parameter to $\beta = 80{,}000$ to accommodate the larger model dimension.

Fig. 14(a) shows the empirical CDF of the required expansion factor $\omega^t_{\min}$. The maximum required expansion factors are approximately 1.64 and 1.35 under the IID and Non-IID settings, respectively, both below the adopted setting $\omega = 2$. Fig. 14(b) further shows that the insufficient-budget rate decreases rapidly as ω increases, reaching zero at $\omega = 1.7$ for IID and $\omega = 1.4$ for Non-IID. Thus, with the adopted $\omega = 2$, no insufficient-budget case is observed under either setting. These results further confirm that the adopted interpolation budget provides sufficient evaluations for sparse polynomial recovery.

E. Blockchain Scalability

We evaluate the blockchain scalability of SPIRIT-FL by varying the number of participating clients from 10 to 200. For each client, SPIRIT-FL requires three on-chain transactions: a Bottom-ℓ sketch submission, a commitment submission, and a verification-value disclosure. During private-point verification, client $\mathcal{U}_j$ batches the evaluations $\{f_j(\gamma_i^t)\}_{i\neq j}$ into a single verification transaction. Therefore, one communication round with N participating clients contains $3N$ on-chain transactions, while the verification payload of each client contains $N-1$ finite-field elements.

We consider two transaction submission strategies.

- **Sequential submission.** Transactions are submitted and confirmed sequentially. This setting measures the blockchain latency without transaction parallelization.
- **Phase-wise concurrent submission.** Transactions from different clients in the same protocol phase are submitted concurrently. The next phase starts after all transactions in the current phase are confirmed.

Gas Consumption and Calldata Size. Fig. 15(a) shows the total gas consumption and calldata size in one communication round. Although the number of transactions grows linearly with N, the verification payload of each client grows with the number of private verification points. Specifically, each client uploads $N-1$ verification evaluations in one batched transaction, resulting in $N(N-1)$ evaluation values per communication round. Consequently, the verification-related calldata grows as $\mathcal{O}(N^2)$, while the sketch and commitment submissions contribute only linear overhead. The gas consumption exhibits a similar growth trend because the verification transactions carry increasing calldata as N increases. The two submission strategies incur the same gas consumption and calldata size because they contain identical on-chain operations and transaction contents.

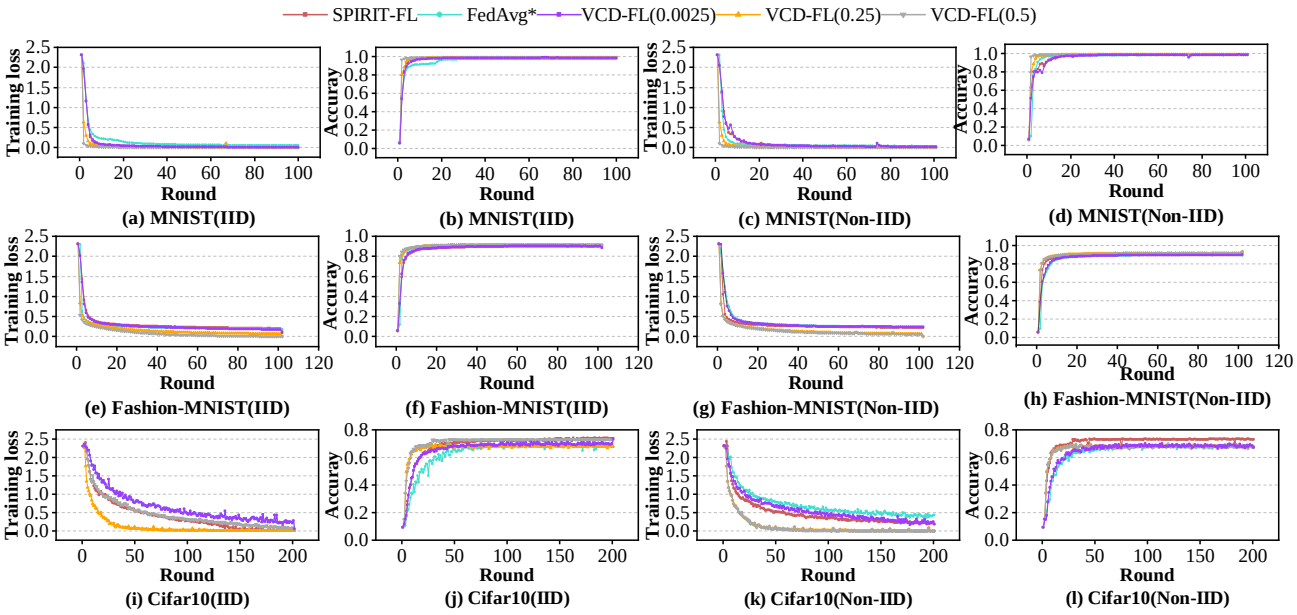


Fig. 16. Additional model utility comparison of SPIRIT-FL, FedAvg*, and VCD-FL on MNIST, Fashion-MNIST, and CIFAR-10 under IID and Non-IID settings.

Round Latency. The blockchain latency of one communication round is defined as

$$L_{\text{round}} = t_{\text{last}}^{\text{confirm}} - t_{\text{first}}^{\text{submit}},$$

where $t_{\text{first}}^{\text{submit}}$ and $t_{\text{last}}^{\text{confirm}}$ denote the submission time of the first transaction and the confirmation time of the last transaction in the round, respectively.

As shown in Fig. 15(b), the latency under sequential submission increases with the number of participating clients because transactions are confirmed one by one. Phase-wise concurrent submission reduces the round latency by allowing transactions within the same protocol phase to be processed concurrently. Thus, the latency is determined by the completion time of each phase rather than the sum of the confirmation times of all transactions.

Effective Transaction Throughput. The effective transaction throughput is calculated as

$$\mathsf{TPS} = \frac{3N}{L_{\text{round}}},$$

where L_{round} is measured in seconds and $3N$ is the total number of transactions in one communication round.

Under sequential submission, the effective throughput remains relatively stable as the number of clients increases, since the number of transactions and the confirmation latency increase simultaneously. Phase-wise concurrent submission achieves higher throughput by processing transactions from different clients concurrently within the same phase. However, batching reduces only the number of verification transactions; it does not eliminate the $N(N-1)$ verification values that must be recorded on-chain.

Overall, SPIRIT-FL maintains a linear number of on-chain transactions through batched verification-value disclosure. The total verification calldata, however, grows quadratically with the number of clients. Phase-wise concurrent submission reduces the round completion latency and improves the effective transaction throughput without changing the on-chain workload for a fixed number of participating clients.

F. Additional Model Utility Results

Following the experimental protocol used in Fig. 9, we further evaluate the model utility of SPIRIT-FL on MNIST, Fashion-MNIST, and CIFAR-10 under both IID and Non-IID settings. As shown in Fig. 16, SPIRIT-FL shows stable convergence across all evaluated datasets and data distributions. On MNIST and Fashion-MNIST, SPIRIT-FL achieves similar model utility despite employing a substantially smaller Top-k sparsification ratio. On CIFAR-10, particularly under the Non-IID setting, SPIRIT-FL achieves lower training loss while maintaining competitive test accuracy. The results show that the sparse polynomial representation and similarity-based weighting mechanism do not noticeably degrade model utility under different datasets and data distributions.